Summarize

Chapter 5. Component-Based Decomposition Patterns

Monday, November 1, 11:53



Addison and Austen chose to use the component-based decomposition approach, but were unsure about the details of each decomposition pattern. They tried to research this approach, but did not find much on the internet about it. Once again, they met with Logan in the conference room for advice on what these patterns are all about and how to use them.

"Listen, Logan," said Addison, "I want to start out by saying we both really appreciate the amount of time you have been spending with us to get this migration process started. I know you're super busy on your own firefights."

"No problem," said Logan. "Us firefighters have to stick together. I've been in your shoes before, so I know what it's like flying blind on these sort of things. Besides, this is a highly visible migration effort, and it's important you both get this thing right the first time. Because there won't be a second time."

"Thanks, Logan," said Austen. "I've got a game in about two hours, so we'll try to make this short. You talked earlier about component-based decomposition, and we chose that approach, but we aren't able to find much about it on the internet."

"I'm not surprised," said Logan. "Not much has been written about them yet, but I know a book is coming out describing these patterns in detail sometime later this year. I first learned about these decomposition patterns at a conference about four years ago in a session with an experienced software architect. I was really impressed with the iterative and methodical approach to safely move from a monolithic architecture to a distributed one like service-based architecture and microservices. Since then I've been using these patterns with quite a bit of success."

"Can you show us how these patterns work?" asked Addison.

"Sure," said Logan. "Let's take it one pattern at a time."

Component-based decomposition (introduced in Chapter 4) is a highly effective technique for breaking apart a monolithic application when the codebase is somewhat structured and grouped by namespaces (or directories). This chapter introduces a set of patterns, known as *component-based decomposition patterns*, that describe the refactoring of monolithic source code to arrive at a set of well-defined components that can eventually become services. These decomposition patterns significantly ease the effort of migrating monolithic applications to distributed architectures.

Figure 5-1 shows the road map for the component-based decomposition patterns described in this chapter and how they are used together to break apart a monolithic application. Initially, these patterns are used together in sequence when moving a monolithic application to a distributed one, and then individually as maintenance is applied to the monolithic application during migration. These decomposition patterns are summarized as follows:

"Identify and Size Components Pattern"

Typically the first pattern applied when breaking apart a monolithic application. This pattern is used to identify, manage, and properly size components.

"Gather Common Domain Components Pattern"

Used to consolidate common business domain logic that might be duplicated across the application, reducing the number of potentially duplicate services in the resulting distributed architecture.

"Flatten Components Pattern"

Used to collapse or expand domains, subdomains, and components, thus ensuring that source code files reside only within well-defined components.

"Determine Component Dependencies Pattern"

Used to identify component dependencies, refine those dependencies, and determine the feasibility and overall level of effort for a migration from a monolithic architecture to a distributed one.

"Create Component Domains Pattern"

Used to group components into logical domains within the application and to refactor component namespaces and/or directories to align with a particular domain.

"Create Domain Services Pattern"

Used to physically break apart a monolithic architecture by moving logical domains within the monolithic application to separately deployed domain services.

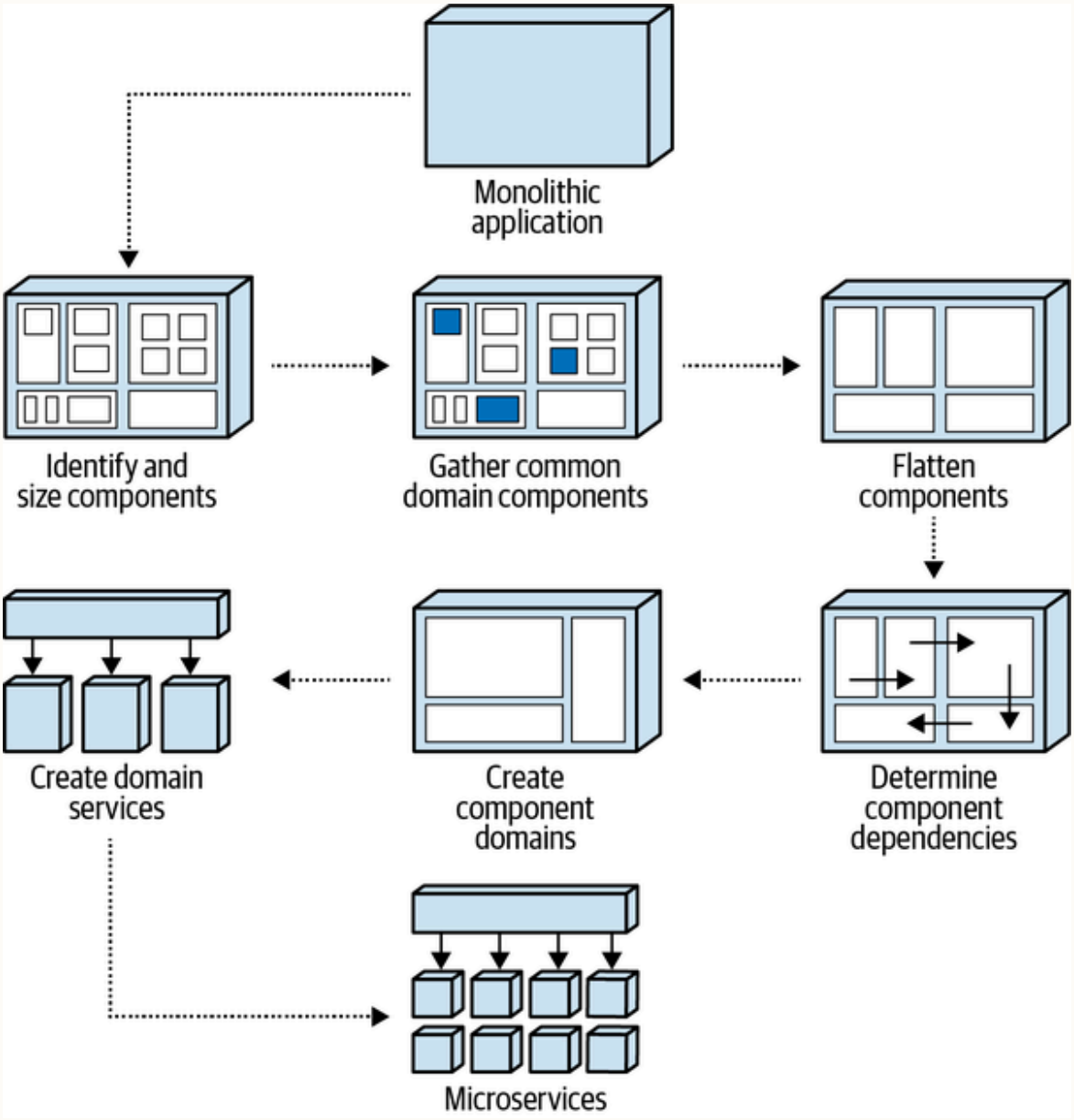


Figure 5-1. Component-based decomposition pattern flow and usage

Each pattern described in this chapter is divided into three sections. The first section, "Pattern Description," describes how the pattern works, why the pattern is important, and what the outcome is of applying the pattern. Knowing that most systems are moving targets during a migration, the second section, "Fitness Functions for Governance," describes the automated governance that can be used after applying the pattern to continually analyze and verify the correctness of the codebase during ongoing maintenance. The third section uses the real-world Sysops Squad application (see "Introducing the Sysops Squad Saga") to illustrate the use of the pattern and illustrate the transformations of the application after the pattern has been applied.

ARCHITECTURE STORIES

Throughout this chapter, we will be using architecture stories as a way of recording and describing code refactoring that impacts the structural aspect of the application for each of the Sysops Squad sagas. Unlike *user stories*, which describe a feature that needs to be implemented or changed, an *architecture story* describes particular code refactoring that impacts the overall structure of an application and satisfies some sort of business driver (such as increased scalability, better time-to-market, etc.). For example, if an architect sees the need to break apart a payment service to support better overall extensibility for adding additional payment types, a new architecture story would be created and read as follows:

As an architect, I need to decouple the payment service to support better extensibility and agility when adding additional payment types.

We view architecture stories as separate from *technical debt* stories. Technical debt stories usually capture things a developer needs to do in a later iteration to "clean up the code," whereas an architecture story captures something that needs to change quickly to support a particular architectural characteristic or business need.

Identify and Size Components Pattern

The first step in any monolithic migration is to apply the *Identify and Size Components* pattern. The purpose of this pattern is to identify and catalog the architectural components (logical building blocks) of the application and then properly size the components.

Pattern Description

Because services are built from components, it is critical to not only identify the components within an application, but to properly size them as well. This pattern is used to identify components that are either too big (doing too much) or too small (not doing enough). Components that are too large relative to other components are generally more coupled to other components, are harder to break into separate services, and lead to a less modular architecture.

Unfortunately, it is difficult to determine the size of a component. The number of source files, classes, and total lines of code are not good metrics because every programmer designs classes, methods, and functions differently. One metric we've found useful for component sizing is calculating the total number of statements within a given component (the sum of statements within all source files contained within a namespace or directory). A *statement* is a single complete action performed in the source code, usually terminated by a

special character (such as a semicolon in languages such as Java, C, C++, C#, Go, and JavaScript; or a newline in languages such as F#, Python, and Ruby). While not a perfect metric, at least it's a good indicator of how much the component is doing and how complex the component is.

Having a relatively consistent component size within an application is important. Generally speaking, the size of components in an application should fall between one to two standard deviations from the average (or mean) component size. In addition, the percentage of code represented by each component should be somewhat evenly distributed between application components and not vary significantly.

While many static code analysis tools can show the number of statements within a source file, many of them don't accumulate total statement by *component*. Because of this, the architect usually must perform manual or automated post-processing to accumulate total statements by component and then calculate the percentage of code that component represents.

Regardless of the tools or algorithms used, the important information and metrics to gather and calculate for this pattern are shown in Table 5-1 and are defined in the following list.

Table 5-1. Component inventory and component size analysis example

Component name	Component namespace	Percent	Statements	Files
Billing Payment	ss.billing.payment	5	4,312	23
Billing History	ss.billing.history	4	3,209	17
Customer Notification	ss.customer.notification	2	1,433	7

Component name

A descriptive name and identifier of the component that is consistent throughout application diagrams and documentation. The component name should be clear enough to be as self-describing as possible. For example, the component Billing History shown in Table 5-1 is clearly a component that contains source code files used to manage a customer's billing history. If the distinct role and responsibility of the component isn't immediately identifiable, consider changing the component (and poten-

tially the corresponding namespace) to a more descriptive one. For example, a component named Ticket Manager leaves too many unanswered questions about its role and responsibility in the system, and should be renamed to better describe its role.

Component namespace

The physical (or logical) identification of the component representing where the source code files implementing that component are grouped and stored. This identifier is usually denoted through a namespace, package structure (Java), or directory structure. When a directory structure is used to denote the component, we usually convert the file separator to a dot (.) and create a corresponding logical namespace. For example, the component namespace for source code files in the *ss/customer/notification* directory structure would have the namespace value `ss.customer.notification`. Some languages require that the namespace match the directory structure (such as Java with a *package*), whereas other languages (such as C# with a *namespace*) do not enforce this constraint. Whatever namespace identifier is used, make sure the type of identifier is consistent across all of the components in the application.

Percent

The relative size of the component based on its percentage of the overall source code containing that component. The percent metric is helpful in identifying components that appear too large or too small in the overall application. This metric is calculated by taking the total number of statements within the source code files representing that component and dividing that number by the total number of statements in the entire codebase of the application. For example, the percent value of 5 for the `ss.billing.payment` component in Table 5-1 means that this component constitutes 5% of the overall codebase.

Statements

The sum of the total number of source code statements in all source files contained within that component. This metric is useful for determining not only the relative size of the components within an application, but also for determining the overall complexity of the component. For example, a seemingly simple single-purpose component named Customer Wishlist might have a total of 12,000 statements, indicating that the processing of wish list items is perhaps more complex than it looks. This metric is also necessary for calculating the percent metric previously described.

Files

The total number of source code files (such as classes, interfaces, types, and so on) that are contained within the component. While this metric has little to do with the

size of a component, it does provide additional information about the component from a class structure standpoint. For example, a component with 18,409 statements and only 2 files is a good candidate for refactoring into smaller, more contextual classes.

When resizing a large component, we recommend using a functional decomposition approach or a domain-driven approach to identify subdomains that might exist within the large component. For example, assume the Sysops Squad application has a Trouble Ticket component containing 22% of the codebase that is responsible for ticket creation, assignment, routing, and completion. In this case, it might make sense to break the single Trouble Ticket component into four separate components (Ticket Creation, Ticket Assignment, Ticket Routing, and Ticket Completion), reducing the percentage of code each component represents, therefore creating a more modular application. If no clear subdomains exist within a large component, then leave the component as is.

Fitness Functions for Governance

Once this decomposition pattern has been applied and components have been identified and sized correctly, it's important to apply some sort of automated governance to identify new components and to ensure components don't get too large during normal application maintenance and create unwanted or unintended dependencies. Automated holistic fitness functions can be triggered during deployment to alert the architect if specified constraints are exceeded (such as the percent metric discussed previously or use of standard deviations to identify outliers).

Fitness functions can be implemented through custom-written code or through the use of open source or COTS tools as part of a CI/CD pipeline. Some of the automated fitness functions that can be used to help govern this decomposition pattern are as follows.

Fitness function: Maintain component inventory

This automated holistic fitness function, usually triggered on deployment through a CI/CD pipeline, helps keep the inventory of components current. It's used to alert an architect of components that might have been added or removed by the development team. Identifying new or removed components is not only critical for this pattern, but for the other decomposition patterns as well. Example 5-1 shows the pseudocode and algorithm for one possible implementation of this fitness function.

Example 5-1. Pseudocode for maintaining component inventory

```
# Get prior component namespaces that are stored in a datastore
LIST prior_list = read_from_datastore()
```

```
# Walk the directory structure, creating namespaces for each complete path
LIST current_list = identify_components(root_directory)

# Send an alert if new or removed components are identified
LIST added_list = find_added(current_list, prior_list)
LIST removed_list = find_removed(current_list, prior_list)
IF added_list NOT EMPTY {
  add_to_datastore(added_list)
  send_alert(added_list)
}
IF removed_list NOT EMPTY {
  remove_from_datastore(removed_list)
  send_alert(removed_list)
}
```

Fitness function: No component shall exceed <some percent> of the overall codebase

This automated holistic fitness function, usually triggered on deployment through a CI/CD pipeline, identifies components that exceed a given threshold in terms of the percentage of overall source code represented by that component, and alerts the architect if any component exceeds that threshold. As mentioned earlier in this chapter, the threshold percentage value will vary depending on the size of the application, but should be set so as to identify significant outliers. For example, for a relatively small application with only 10 components, setting the percentage threshold to something like 30% would sufficiently identify a component that is too large, whereas for a large application with 50 components, a threshold of 10% would be more appropriate. Example 5-2 shows the pseudocode and algorithm for one possible implementation of this fitness function.

Example 5-2. Pseudocode for maintaining component size based on percent of code

```
# Walk the directory structure, creating namespaces for each complete path
LIST component_list = identify_components(root_directory)

# Walk through all of the source code to accumulate total statements
total_statements = accumulate_statements(root_directory)

# Walk through the source code for each component, accumulating statements
# and calculating the percentage of code each component represents. Send
# an alert if greater than 10%
FOREACH component IN component_list {
  component_statements = accumulate_statements(component)
  percent = component_statements / total_statements
```

```
  IF percent > .10 {
    send_alert(component, percent)
  }
}
```

Fitness function: No component shall exceed <some number of standard deviations> from the mean component size

This automated holistic fitness function, usually triggered on deployment through a CI/CD pipeline, identifies components that exceed a given threshold in terms of the number of standard deviations from the mean of all component sizes (based on the total number of statements in the component), and alerts the architect if any component exceeds that threshold.

Standard deviation is a useful means of determining outliers in terms of component size. Standard deviation is calculated as follows:

$$s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \overline{x})^2}$$

where *N* is the number of observed values, x_i is the observed values, and $\overline{x}$ is the mean of the observed values. The mean of observed values ($\overline{x}$) is calculated as follows:

$$\overline{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$$

The standard deviation can then be used along with the difference from the mean to determine the number of standard deviations the component size is from the mean. Example 5-3 shows the pseudocode for this fitness function, using three standard deviations from the mean as a threshold.

Example 5-3. Pseudocode for maintaining component size based on number of standard deviations

```
# Walk the directory structure, creating namespaces for each complete path
LIST component_list = identify_components(root_directory)

# Walk through all of the source code to accumulate total statements and number
# of statements per component
SET total_statements TO 0
MAP component_size_map
FOREACH component IN component_list {
  num_statements = accumulate_statements(component)
  ADD num_statements TO total_statements
```

```
  ADD component,num_statements TO component_size_map
}

# Calculate the standard deviation
SET square_diff_sum TO 0
num_components = get_num_entries(component_list)
mean = total_statements / num_components
FOREACH component,size IN component_size_map {
  diff = size - mean
  ADD square(diff) TO square_diff_sum
}
std_dev = square_root(square_diff_sum / (num_components - 1))

# For each component calculate the number of standard deviations from the
# mean. Send an alert if greater than 3
FOREACH component,size IN component_size_map {
  diff_from_mean = absolute_value(size - mean);
  num_std_devs = diff_from_mean / std_dev
  IF num_std_devs > 3 {
    send_alert(component, num_std_devs)
  }
}
```

Sysops Squad Saga: Sizing Components

Tuesday, November 2, 09:12



After the discussion with Logan (the lead architect) about component-based decomposition patterns, Addison decided to apply the Identify and Size Components pattern to identify all of the components in the Sysops Squad ticketing application and calculate the size of each component based on the total number of statements in each component.

Addison gathered all the necessary component information and put this information into Table 5-2, calculating the percentage of code for each component based on the total number of statements in the entire application (in this case, 82,931 statements).

Table 5-2. Component size analysis for the Sysops Squad application

Component name	Component namespace	Percent	Statements	Files
Login	ss.login	2	1865	3
Billing Payment	ss.billing.payment	5	4,312	23
Billing History	ss.billing.history	4	3,209	17
Customer Notification	ss.customer.notification	2	1,433	7
Customer Profile	ss.customer.profile	5	4,012	16
Expert Profile	ss.expert.profile	6	5,099	32
KB Maint	ss.kb.maintenance	2	1,701	14
KB Search	ss.kb.search	3	2,871	4
Reporting	ss.reporting	**33**	**27,765**	**162**
Ticket	ss.ticket	8	7,009	45
Ticket Assign	ss.ticket.assign	9	7,845	14
Ticket Notify	ss.ticket.notify	2	1,765	3
Ticket Route	ss.ticket.route	2	1,468	4
Support Contract	ss.supportcontract	5	4,104	24
Survey	ss.survey	3	2,204	5

Component name	Component namespace	Percent	Statements	Files
Survey Notify	ss.survey.notify	2	1,299	3
Survey Templates	ss.survey.templates	2	1,672	7
User Maintenance	ss.users	4	3,298	12

Addison noticed that most of the components listed in Table 5-2 are about the same size, with the exception of the Reporting component (`ss.reporting`) which consisted of 33% of the codebase. Since the Reporting component was significantly larger than the other components (illustrated in Figure 5-2), Addison chose to break this component apart to reduce its overall size.

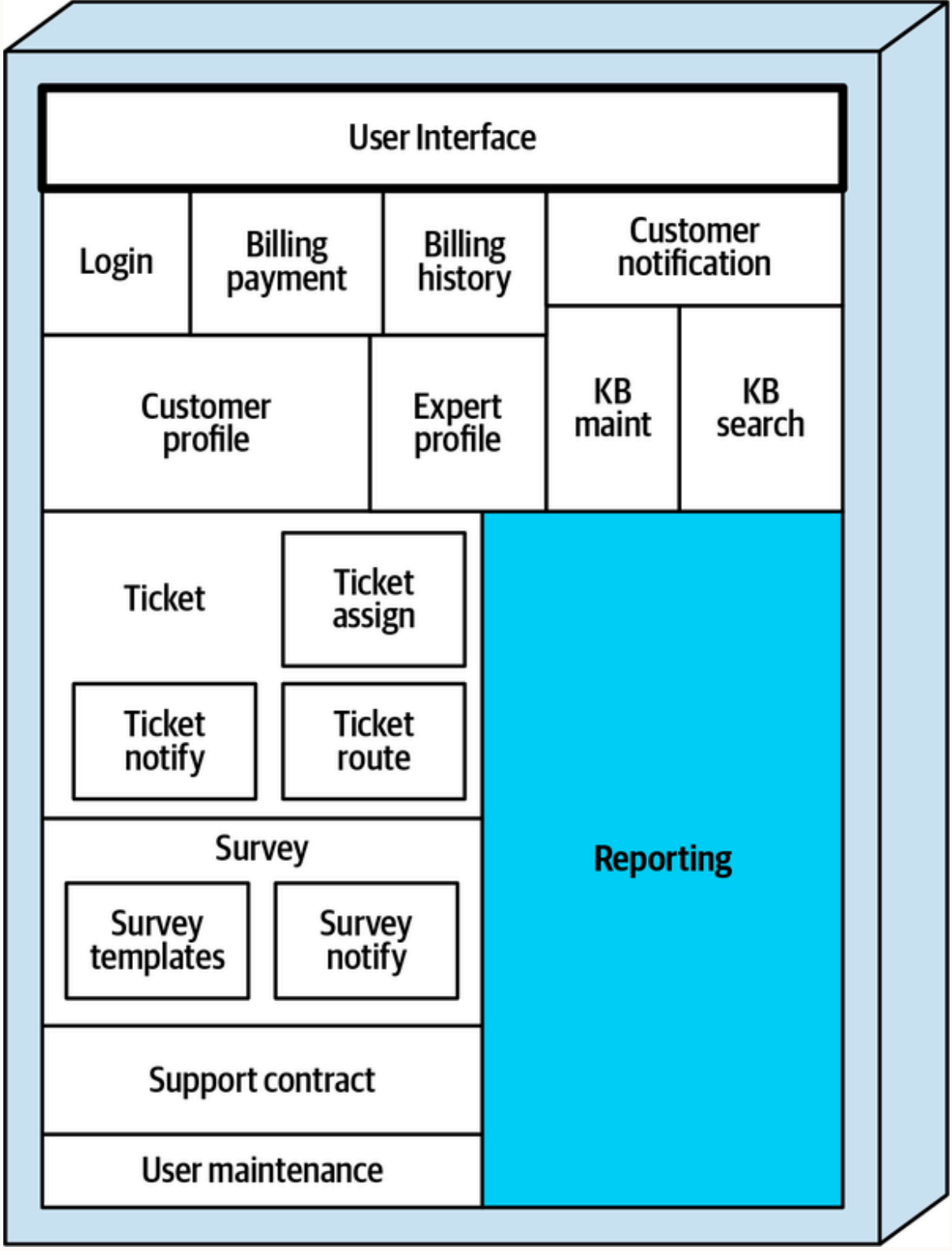


Figure 5-2. The Reporting component is too big and should be broken apart

After doing some analysis, Addison found that the reporting component contained source code that implemented three categories of reports:

- Ticketing reports (ticket demographics reports, tickets per day/week/month reports, ticket resolution time reports, and so on)
- Expert reports (expert utilization reports, expert distribution reports, and so on)
- Financial reports (repair cost reports, expert cost reports, profit reports, and so on)

Addison also identified common (shared) code that all reporting categories used, such as common utilities, calculators, shared data queries, report distribution, and shared data formatters. Addison created an architecture story (see “Architecture Stories”) for this refac-

toring and explained it to the development team. Sydney, one of the Sysops Squad developers assigned the architecture story, refactored the code to break apart the single Reporting component into four separate components—a Reporting Shared component containing the common code and three other components (Ticket Reports, Expert Reports, and Financial Reports), each representing a functional reporting area, as illustrated in Figure 5-3.

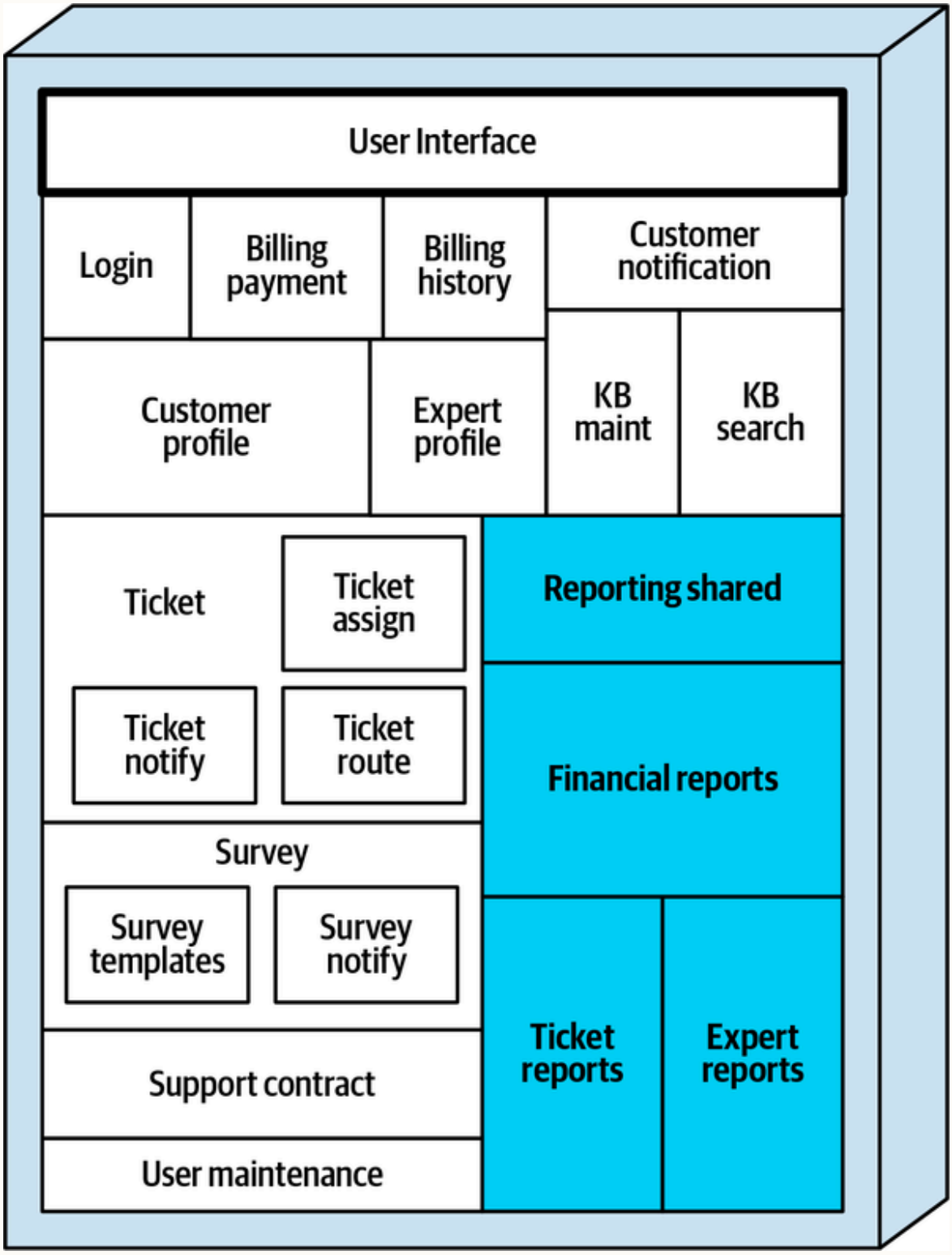


Figure 5-3. The large Reporting component broken into smaller reporting components

After Sydney committed the changes, Addison reanalyzed the code and verified that all of the components were now fairly equally distributed in size. Addison recorded the results of applying this decomposition pattern in Table 5-3.

Table 5-3. Component size after applying the Identify and Size Components pattern

Component name	Component namespace	Percent	Statements	Files
Login	ss.login	2	1865	3
Billing Payment	ss.billing.payment	5	4,312	23
Billing History	ss.billing.history	4	3,209	17
Customer Notification	ss.customer.notification	2	1,433	7
Customer Profile	ss.customer.profile	5	4,012	16
Expert Profile	ss.expert.profile	6	5,099	32
KB Maint	ss.kb.maintenance	2	1,701	14
KB Search	ss.kb.search	3	2,871	4
Reporting Shared	ss.reporting.shared	**7**	**5,309**	**20**
Ticket Reports	ss.reporting.tickets	**8**	**6,955**	**58**
Expert Reports	ss.reporting.experts	**9**	**7,734**	**48**
Financial Reports	ss.reporting.financial	**9**	**7,767**	**36**
Ticket	ss.ticket	8	7,009	45

Component name	Component namespace	Percent	Statements	Files
Ticket Assign	ss.ticket.assign	9	7,845	14
Ticket Notify	ss.ticket.notify	2	1,765	3
Ticket Route	ss.ticket.route	2	1,468	4
Support Contract	ss.supportcontract	5	4,104	24
Survey	ss.survey	3	2,204	5
Survey Notify	ss.survey.notify	2	1,299	3
Survey Templates	ss.survey.templates	2	1,672	7
User Maintenance	ss.users	4	3,298	12

Notice in the preceding Sysops Squad Saga that Reporting no longer exists as a component in Table 5-3 or Figure 5-3. Although the *namespace* still exists (`ss.reporting`), it is no longer considered a component, but rather a subdomain. The refactored components listed in Table 5-3 will be used when applying the next decomposition pattern, Gather Common Domain Components.

Gather Common Domain Components Pattern

When moving from a monolithic architecture to a distributed one, it is often beneficial to identify and consolidate common domain functionality to make common services easier to identify and create. The *Gather Common Domain Components* pattern is used to identify and collect common domain logic and centralize it into a single component.

Pattern Description

Shared *domain* functionality is distinguished from shared *infrastructure* functionality in that domain functionality is part of the business processing logic of an application (such as notification, data formatting, and data validation) and is common to only some processes, whereas infrastructure functionality is operational in nature (such as logging, metrics gathering, and security) and is common to all processes.

Consolidating common domain functionality helps eliminate duplicate services when breaking apart a monolithic system. Often there are only very subtle differences among common domain functionality that is duplicated throughout the application, and these differences can be easily resolved within a single common service (or shared library).

Finding common domain functionality is mostly a manual process, but some automation can be used to assist in this effort (see "Fitness Functions for Governance"). One hint that common domain processing exists in the application is the use of shared classes across components or a common inheritance structure used by multiple components. Take, for example, a class file named *SMTPConnection* in a large codebase that is used by five classes, all contained within different namespaces (components). This scenario is a good indication that common email notification functionality is spread throughout the application and might be a good candidate for consolidation.

Another way of identifying common domain functionality is through the name of a logical component or its corresponding namespace. Consider the following components (represented as namespaces) in a large codebase:

- Ticket Auditing (`penultimate.ss.ticket.audit`)
- Billing Auditing (`penultimate.ss.billing.audit`)
- Survey Auditing (`penultimate.ss.survey.audit`)

Notice how each of these components (Ticket Auditing, Billing Auditing, and Survey Auditing) all have the same thing in common—writing the action performed and the user requesting the action to an audit table. While the context may be different, the final outcome is the same—inserting a row in an audit table. This common domain functionality can be consolidated into a new component called `penultimate.ss.shared.audit` , resulting in less duplication of code and also fewer services in the resulting distributed architecture.

Not all common domain functionality necessarily becomes a shared service. Alternatively, common code could be gathered into a *shared library* that is bound to the code during compile time. The pros and cons of using a shared service rather than a shared library are discussed in detail in Chapter 8.

Fitness Functions for Governance

Automating the governance of shared domain functionality is rather difficult because of the subjectiveness of identifying shared functionality and classifying it as domain functionality versus infrastructure functionality. For the most part, the fitness functions used to govern this pattern are therefore somewhat manual. That said, there are some ways to automate the governance to assist in the manual interpretation of common domain functionality. The following fitness functions can assist in finding common domain functionality.

Fitness function: Find common names in leaf nodes of component namespace

This automated holistic fitness function can be triggered on deployment through a CI/CD pipeline to locate common names within the namespace of a component. When a common ending namespace node name is found between two or more components, the architect is alerted and can analyze the functionality to determine if it is common domain logic. So that the same alert isn't continuously sent as a "false positive," an exclusion file can be used to store those namespaces that have common ending node names but are not deemed common domain logic (such as multiple namespaces ending in `.calculate` or `.validate`). Example 5-4 shows the pseudocode for this fitness function.

Example 5-4. Pseudocode for finding common namespace leaf node names

```
# Walk the directory structure, creating namespaces for each complete path
LIST component_list = identify_components(root_directory)

# Locate possible duplicate component node names that are not in the exclusion
# list stored in a datastore
LIST excluded_leaf_node_list = read_datastore()
LIST leaf_node_list
LIST common_component_list
FOREACH component IN component_list {
  leaf_name = get_last_node(component)
  IF leaf_name IN leaf_node_list AND
     leaf_name NOT IN excluded_leaf_node_list {
    ADD component TO common_component_list
  } ELSE {
    ADD leaf_name TO leaf_node_list
  }
}

# Send an alert if any possible common components were found
IF common_component_list NOT EMPTY {
  send_alert(common_component_list)
}
```

Fitness function: Find common code across components

This automated holistic fitness function can be triggered on deployment through a CI/CD pipeline to locate common classes used between namespaces. While not always accurate, it does help in alerting an architect of possible duplicate domain functionality. Like the previous fitness function, an exclusion file is used to reduce the number of "false positives" for known common code that is not considered duplicate domain logic. Example 5-5 shows the pseudocode for this fitness function.

Example 5-5. Pseudocode for finding common source files between components

```
# Walk the directory structure, creating namespaces for each complete path and a list
# of source file names for each component
LIST component_list = identify_components(root_directory)
LIST source_file_list = get_source_files(root_directory)
MAP component_source_file_map
FOREACH component IN component_list {
  LIST component_source_file_list = get_source_files(component)
  ADD component, component_source_file_list TO component_source_file_map
}

# Locate possible common source file usage across components that are not in
# the exclusion list stored in a datastore
LIST excluded_source_file_list = read_datastore()
LIST common_source_file_list
FOREACH source_file IN source_file_list {
  SET count TO 0
  FOREACH component,component_source_file_list IN component_source_file_map {
    IF source_file IN component_source_file_list {
      ADD 1 TO count
    }
  }
  IF count > 1 AND source_file NOT IN excluded_source_file_list {
    ADD source_file TO common_source_file_list
  }
}

# Send an alert if any source files are used in multiple components
IF common_source_file_list NOT EMPTY {
       send_alert(common_source_file_list)
}
```

Sysops Squad Saga: Gathering Common Components

Friday, November 5, 10:34



Having identified and sized the components in the Sysops Squad application, Addison applied the Gather Common Domain Components pattern to see if any common functionality existed between components. From the list of components in Table 5-3, Addison noticed there were three components all related to notifying a Sysops Squad customer, and listed these in Table 5-4.

Table 5-4. Sysops Squad components with common domain functionality

Component	Namespace	Responsibility
Customer Notification	ss.customer.notification	General notification
Ticket Notify	ss.ticket.notify	Notify that expert is en route
Survey Notify	ss.survey.notify	Send survey email

While each of these notification components had a different context for notifying a customer, Addison realized they all have one thing in common—they all sent information to a customer. Figure 5-4 illustrates these common notification components within the Sysops Squad application.

Noticing that the source code contained in these components was also very similar, Addison consulted with Austen (the other Sysops Squad architect). Austen liked the idea of a single notification component, but was concerned about impacting the overall level of coupling between components. Addison agreed that this might be an issue and investigated this trade-off further.

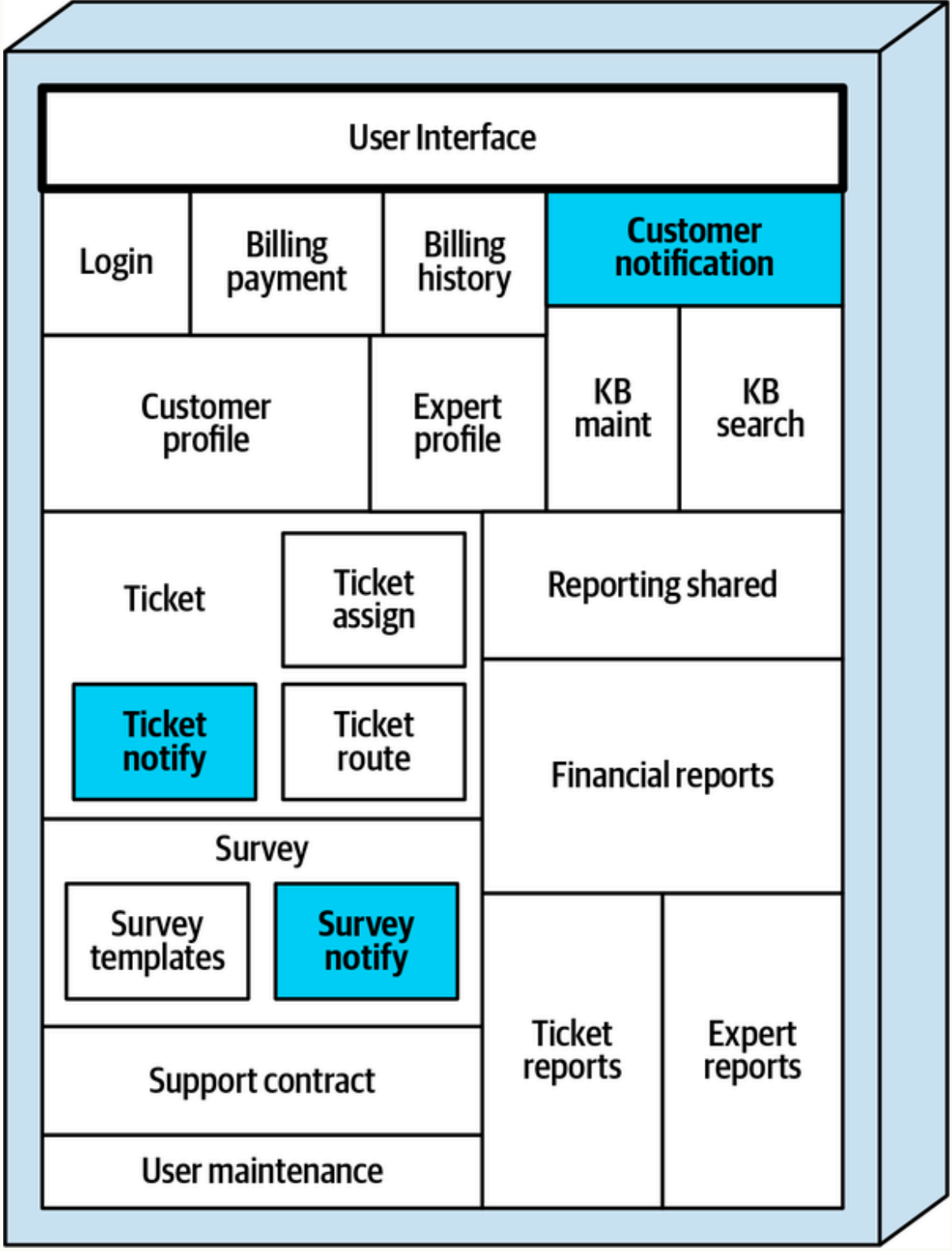


Figure 5-4. Notification functionality is duplicated throughout the application

Addison analyzed the incoming (afferent) coupling level for the existing Sysops Squad notification components and came up with the resulting coupling metrics listed in Table 5-5, with "CA" representing the number of other components requiring that component (afferent coupling).

Table 5-5. Sysops Squad coupling analysis before component consolidation

Component	CA	Used by
Customer Notification	2	Billing Payment, Support Contract
Ticket Notify	2	Ticket, Ticket Route
Survey Notify	1	Survey

Addison then found that if the customer notification functionality was consolidated into a single component, the coupling level for the resulting single component increased to an incoming coupling level of 5, as shown in Table 5-6.

Table 5-6. Sysops Squad coupling analysis after component consolidation

Component	CA	Used by
Notification	5	Billing Payment, Support Contract, Ticket, Ticket Route, Survey

Addison brought these findings to Austen, and they discussed the results. What they found is that, while the new consolidated component had a fairly high level of incoming coupling, it didn't affect the overall afferent (incoming) coupling level for notifying a customer. In other words, the three separate components had a total incoming coupling level of 5, but so did the single consolidated component.

Addison and Austen both realized how important it was to analyze the coupling level after consolidating common domain functionality. In some cases, combining common domain functionality into a single consolidated component increased the incoming coupling level of that component, thus resulting in too many dependencies on a single shared component within the application. However, in this case both Addison and Austen were comfortable with the coupling analysis, and agreed to consolidate the notification functionality to reduce the duplication of both code and functionality.

Addison wrote an architecture story to combine all of the notification functionality into a single namespace representing a common Notification component. Sydney, assigned to the architecture story, refactored the source code, creating a single component for customer notification, as illustrated in Figure 5-5.

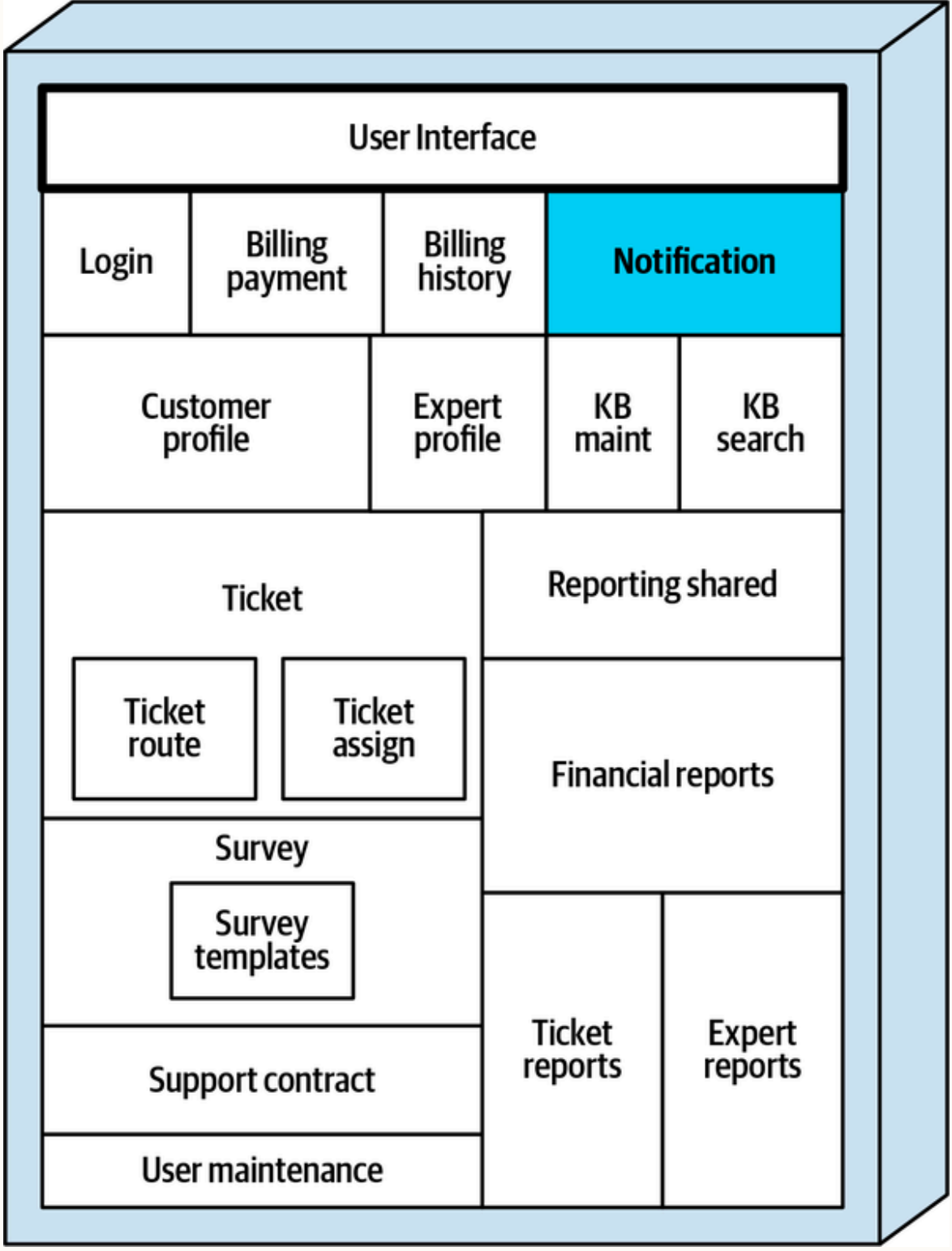


Figure 5-5. Notification functionality is consolidated into a new single component called Notification

Table 5-7 shows the resulting components after Sydney implemented the architecture story Addison created. Notice that the Customer Notification component (`ss.customer.notification`), Ticket Notify component (`ss.ticket.notify`), and Survey Notify components (`ss.survey.notify`) were removed, and the source code moved to the new consolidated Notification component (`ss.notification`).

Table 5-7. Sysops Squad components after applying the Gather Common Domain Components pattern

Component	Namespace	Responsibility
Login	ss.login	User and customer login
Billing Payment	ss.billing.payment	Customer monthly billing
Billing History	ss.billing.history	Payment history
Customer Profile	ss.customer.profile	Maintain customer profile
Expert Profile	ss.expert.profile	Maintain expert profile
KB Maint	ss.kb.maintenance	Maintain & view knowledge base
KB Search	ss.kb.search	Search knowledge base
Notification	ss.notification	**All customer notification**
Reporting Shared	ss.reporting.shared	Shared functionality
Ticket Reports	ss.reporting.tickets	Create ticketing reports
Expert Reports	ss.reporting.experts	Create expert reports
Financial Reports	ss.reporting.financial	Create financial reports
Ticket	ss.ticket	Ticket creation & maintenance
Ticket Assign	ss.ticket.assign	Assign expert to ticket
Ticket Route	ss.ticket.route	Send ticket to expert
Support Contract	ss.supportcontract	Support contract maintenance
Survey	ss.survey	Send and receive surveys
Survey Templates	ss.survey.templates	Maintain survey templates
User Maintenance	ss.users	Maintain internal users

Flatten Components Pattern

As mentioned previously, components—the building blocks of an application—are usually identified through namespaces, package structures, or directory structures and are implemented through class files (or source code files) contained within these structures. However, when components are built on top of other components, which are in turn built on top of other components, they start to lose their identity and stop becoming components as per our definition. The *Flatten Components* pattern is used to ensure that components are not built on top of one another, but rather flattened and represented as leaf nodes in a directory structure or namespace.

Pattern Description

When the namespace representing a particular component gets extended (in other words, another node is added to the namespace or directory structure), the prior namespace or directory no longer represents a component, but rather a subdomain. To illustrate this point, consider the customer survey functionality within the Sysops Squad application represented by two components: Survey (`ss.survey`) and Survey Templates (`ss.survey.templates`). Notice in Table 5-8 how the `ss.survey` namespace, which contains five class files used to manage and collect the surveys, is extended with the `ss.survey.templates` namespace to include seven classes representing each survey type send out to customers.

Table 5-8. The Survey component contains orphaned classes and should be flattened

Component name	Component namespace	Files
→ Survey	`ss.survey`	5
Survey Templates	ss.survey.templates	7

While this structure might seem to make sense from a developer's standpoint in order to keep the template code separate from survey processing, it does create some problems because Survey Templates, as a component, would be considered part of the Survey component. One might be tempted to consider Survey Templates as a *subcomponent* of Survey, but then issues arise when trying to form services from these components—should both components reside in a single service called Survey, or should the Survey Templates be a separate service from the Survey service?

We've resolved this dilemma by defining a component as the last node (or leaf node) of the namespace or directory structure. With this definition, `ss.survey.templates` is a compo-

nent, whereas `ss.survey` would be considered a *subdomain*, not a component. We further define namespaces such as `ss.survey` as *root namespaces* because they are extended with other namespace nodes (in this case, `.templates`).

Notice how the `ss.survey` root namespace in Table 5-8 contains five class files. We call these class files *orphaned classes* because they do not belong to any definable component. Recall that a component is identified by a leaf node namespace containing source code. Because the `ss.survey` namespace was extended to include `.templates`, `ss.survey` is no longer considered a component and therefore should not contain any class files.

The following terms and corresponding definitions are important for understanding and applying the Flatten Components decomposition pattern:

Component

A collection of classes grouped within a *leaf node namespace* that performs some sort of specific functionality in the application (such as payment processing or customer survey functionality).

Root namespace

A namespace node that has been extended by another namespace node. For example, given the namespaces `ss.survey` and `ss.survey.templates`, `ss.survey` would be considered a root namespace because it is extended by `.templates`. Root namespaces are also sometimes referred to as *subdomains*.

Orphaned classes

Classes contained within a root namespace, and hence have no definable component associated with them.

These definitions are illustrated in Figure 5-6, where the box with a C represents source code contained within that namespace. This diagram (and all others like it) are purposely drawn from the bottom up to emphasize the notion of *hills* in the application, as well as emphasize the notion of namespaces *building upon* each other.

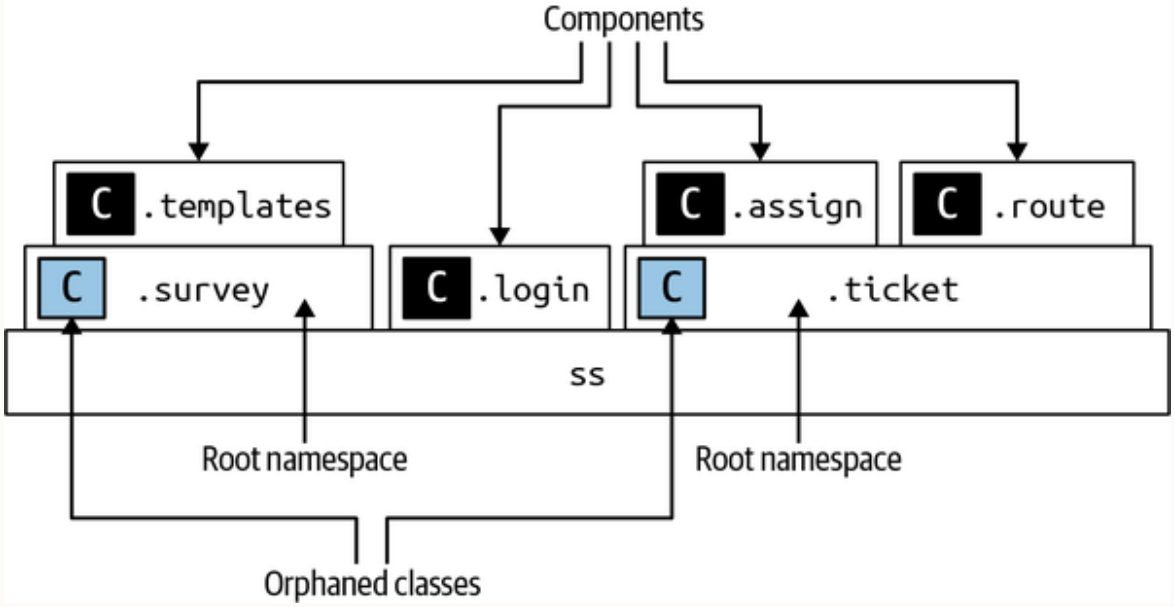


Figure 5-6. Components, root namespaces, and orphaned classes (C box denotes source code)

Notice that since both `ss.survey` and `ss.ticket` are extended through other namespace nodes, those namespaces are considered root namespaces, and the classes contained in those root namespaces are hence orphaned classes (belonging to no defined component). Thus, the only components denoted in Figure 5-6 are `ss.survey.templates`, `ss.login`, `ss.ticket.assign`, and `ss.ticket.route`.

The Flatten Components decomposition pattern is used to move orphaned classes to create well-defined components that exist only as leaf nodes of a directory or namespace, creating well-defined subdomains (root namespaces) in the process. We refer to the *flattening* of components as the breaking down (or building up) of namespaces within an application to remove orphaned classes. For example, one way of flattening the `ss.survey` root namespace in Figure 5-6 and remove orphaned classes is to move the source code contained in the `ss.survey.templates` namespace down to the `ss.survey` namespace, thereby making `ss.survey` a single component (`.survey` is now the leaf node of that namespace). This flattening option is illustrated in Figure 5-7.

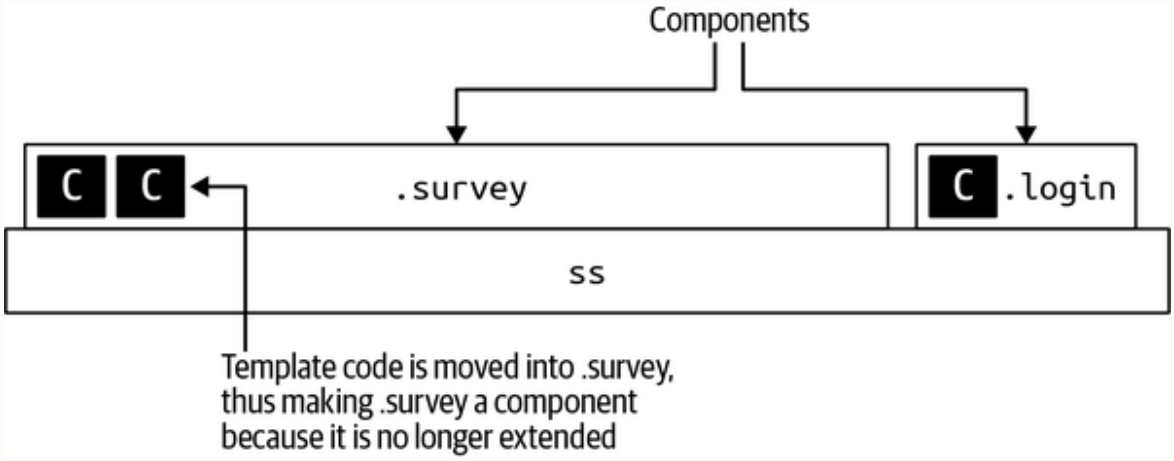


Figure 5-7. Survey is flattened by moving the survey template code into the .survey namespace

Alternatively, flattening could also be applied by taking the source code in `ss.survey` and applying functional decomposition or domain-driven design to identify separate functional areas within the root namespace, thus forming components from those functional areas. For example, suppose the functionality within the `ss.survey` namespace creates and sends a survey to a customer, and then processes a completed survey received from the customer. Two components could be created from the `ss.survey` namespace: `ss.survey.create`, which creates and sends the survey, and `ss.survey.process`, which processes a survey received from a customer. This form of flattening is illustrated in Figure 5-8.

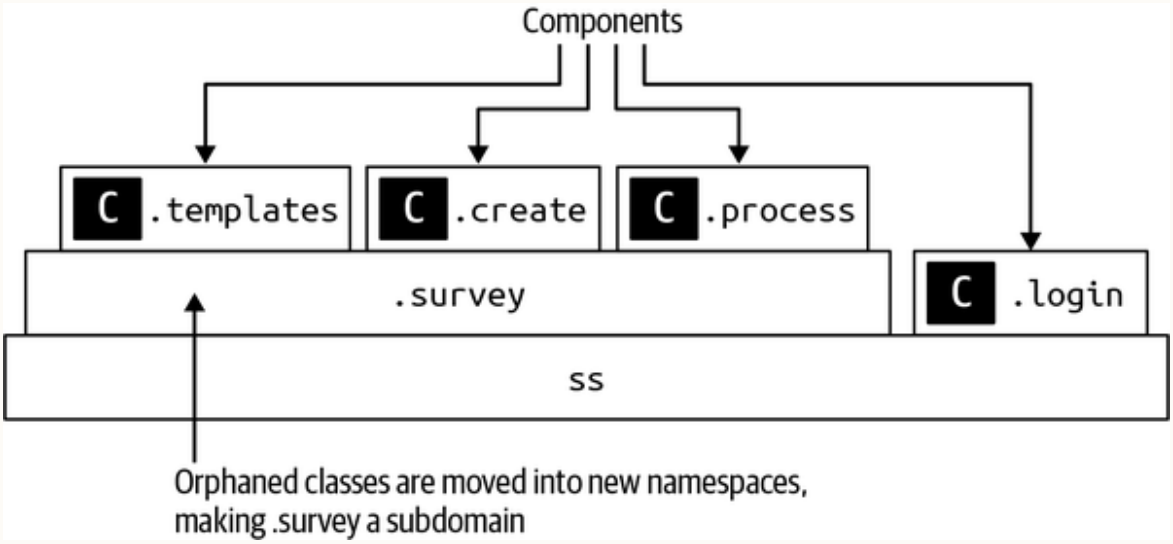


Figure 5-8. Survey is flattened by moving the orphaned classes to new leaf nodes (components)

TIP

Regardless of the direction of flattening, make sure source code files reside only in leaf node namespaces or directories so that source code can always be identified within a specific component.

Another common scenario where orphaned source code might reside in a root namespace is when code is shared by other components within that namespace. Consider the example in Figure 5-9 where customer survey functionality resides in three components (`ss.survey.templates`, `ss.survey.create`, and `ss.survey.process`), but common code (such as interfaces, abstract classes, common utilities) resides in the root namespace `ss.survey`.

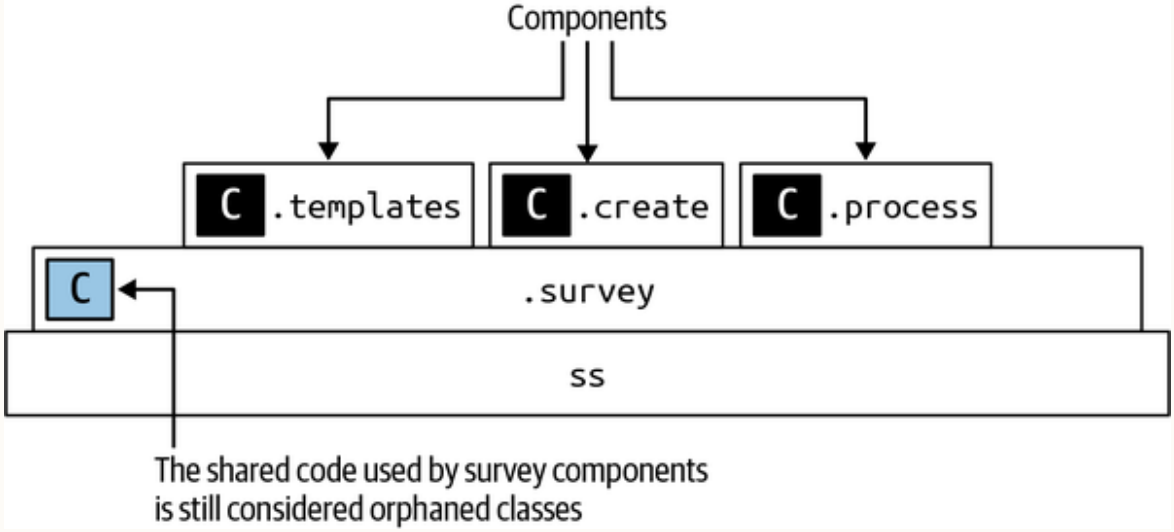


Figure 5-9. Shared code in .survey is considered orphaned classes and should be moved

The shared classes in `ss.survey` would still be considered orphaned classes, even though they represent shared code. Applying the Flatten Components pattern would move those shared orphaned classes to a new component called `ss.survey.shared`, therefore removing all orphaned classes from the `ss.survey` subdomain, as illustrated in Figure 5-10.

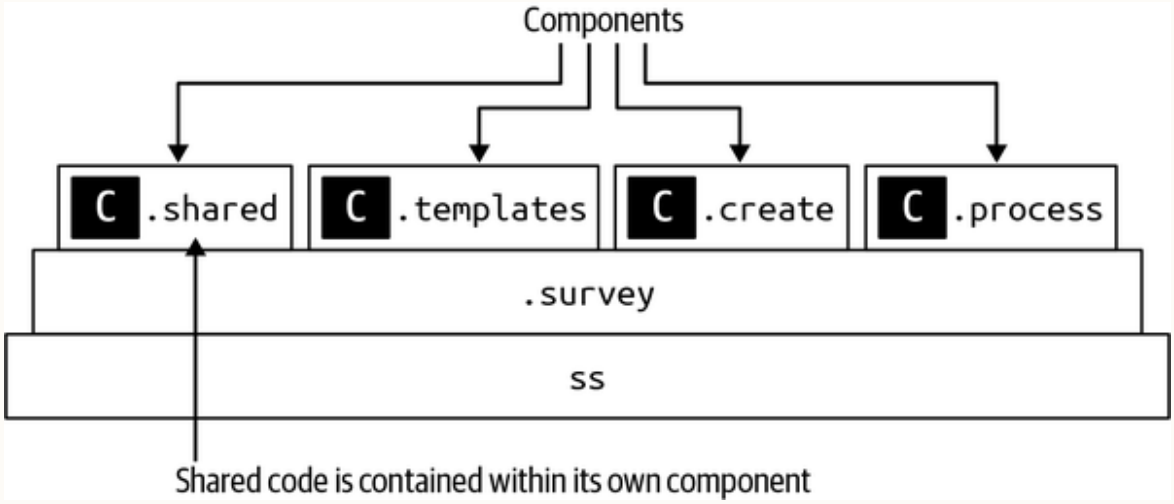


Figure 5-10. Shared survey code is moved into its own component

Our advice when moving shared code to a separate component (leaf node namespace) is to pick a word that is not used in any existing codebase in the domain, such as `.sharedcode`, `.commoncode`, or some such unique name. This allows the architect to generate metrics based on the number of shared components in the codebase, as well as the *percentage* of source code that is shared in the application. This is a good indicator as to the feasibility of breaking up the monolithic application. For example, if the sum of all the statements in all namespaces ending with `.sharedcode` constitutes 45% of the overall source code, chances are moving to a distributed architecture will result in too many shared libraries and end up becoming a nightmare to maintain because of shared library dependencies.

Another good metric involving the analysis of shared code is the *number* of components ending in `.sharedcode` (or whatever common shared namespace node is used). This metric

gives the architect insight into how many shared libraries (JAR, DLL, and so on) or shared services will result from breaking up the monolithic application.

Fitness Functions for Governance

Applying the Flatten Components decomposition pattern involves a fair amount of subjectivity. For example, should code from leaf nodes be consolidated into the root namespace, or should code in a root namespace be moved into leaf nodes? That said, the following fitness function can assist in automating the governance of keeping components flat (only in leaf nodes).

Fitness function: No source code should reside in a root namespace

This automated holistic fitness function can be triggered on deployment through a CI/CD pipeline to locate orphaned classes—classes that reside in a root namespace. Use of this fitness function helps keep components flat when undergoing a monolithic migration, especially when performing ongoing maintenance to the monolithic application during the migration effort. Example 5-6 shows the pseudocode that alerts an architect when orphaned classes appear anywhere in the codebase.

Example 5-6. Pseudocode for finding code in root namespaces

```
# Walk the directory structure, creating namespaces for each complete path
LIST component_list = identify_components(root_directory)

# Send an alert if a non-leaf node in any component contains source files
FOREACH component IN component_list {
  LIST component_node_list = get_nodes(component)
  FOREACH node IN component_node_list {
    IF contains_code(node) AND NOT last_node(component_node_list) {
      send_alert(component)
    }
  }
}
```

Sysops Squad Saga: Flattening Components

Wednesday, November 10, 11:10



After applying the "Gather Common Domain Components Pattern", Addison analyzed the results in Table 5-7 and observed that the Survey and Ticket components

contained orphaned classes. Addison highlighted these components in Table 5-9 and in Figure 5-11.

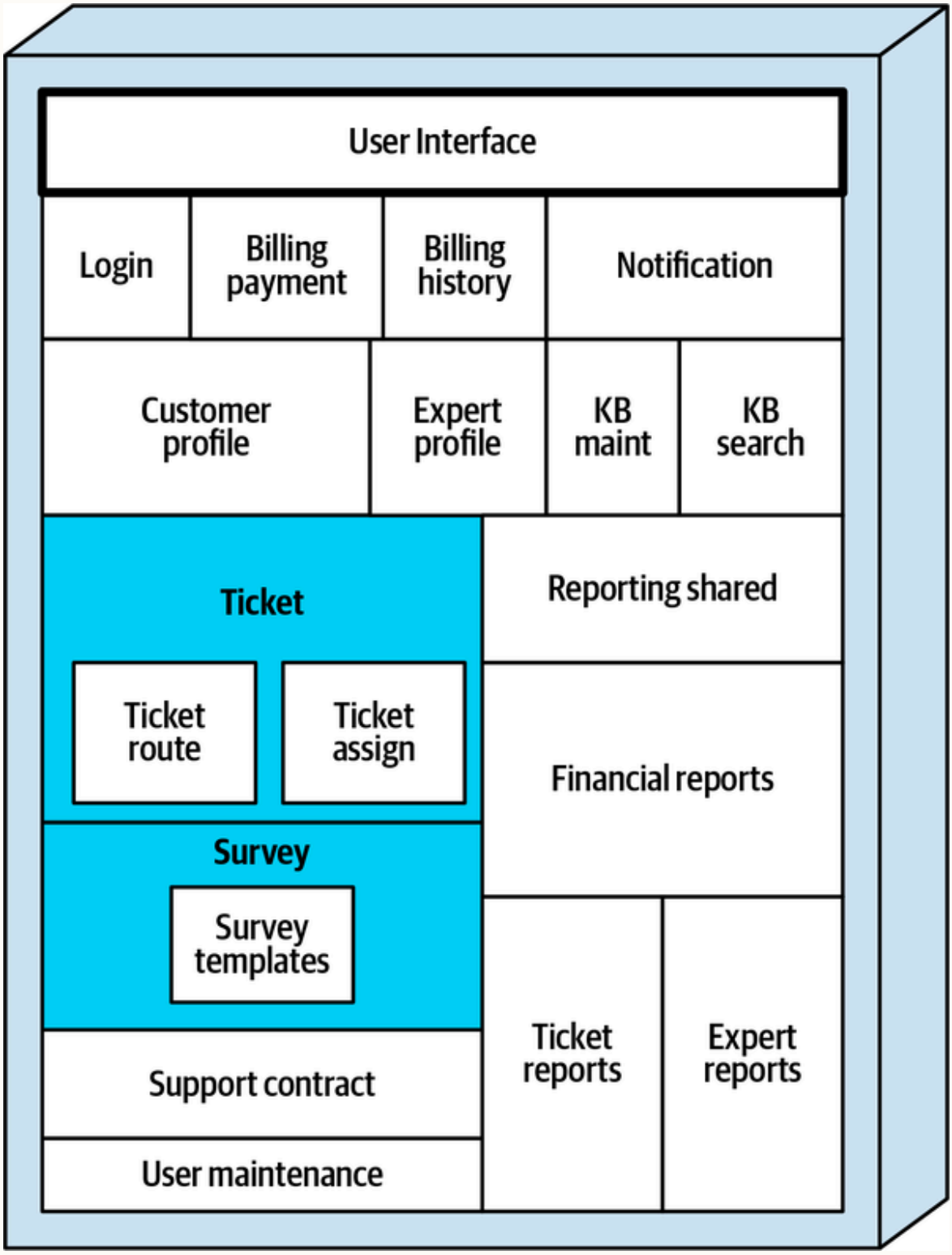


Figure 5-11. The Survey and Ticket components contain orphaned classes and should be flattened

Table 5-9. Sysops Squad Ticket and Survey components should be flattened

Component name	Component namespace	Statements	Files
Ticket	ss.ticket	**7,009**	**45**
Ticket Assign	ss.ticket.assign	7,845	14
Ticket Route	ss.ticket.route	1,468	4
Survey	ss.survey	**2,204**	**5**
Survey Templates	ss.survey.templates	1,672	7

Addison decided to address the ticketing components first. Knowing that flattening components meant getting rid of source code in nonleaf nodes, Addison had two choices: consolidate the code contained in the ticket assignment and ticket routing components into the `ss.ticket` component, or break up the 45 classes in the `ss.ticket` component into separate components, thus making `ss.ticket` a subdomain. Addison discussed these options with Sydney (one of the Sysops Squad developers), and based on the complexity and frequent changes in the ticket assignment functionality, decided to keep those components separate and move the orphaned code from the `ss.ticket` root namespace into other namespaces, thus forming new components.

With help from Sydney, Addison found that the 45 orphaned classes contained in the `ss.ticket` namespace implemented the following ticketing functionality:

- Ticket creation and maintenance (creating a ticket, updating a ticket, canceling a ticket, etc.)
- Ticket completion logic
- Shared code common to most of the ticketing functionality

Since ticket assignment and ticket routing functionality were already in their own components (`ss.ticket.assign` and `ss.ticket.route`, respectively), Addison created an architecture story to move the source code contained in the `ss.ticket` namespace to three new components, as shown in Table 5-10.

Table 5-10. The prior Sysops Squad Ticket component broken into three new components

Component	Namespace	Responsibility
Ticket Shared	ss.ticket.shared	Common code and utilities
Ticket Maintenance	ss.ticket.maintenance	Add and maintain tickets
Ticket Completion	ss.ticket.completion	Complete ticket and initiate survey
Ticket Assign	ss.ticket.assign	Assign expert to ticket
Ticket Route	ss.ticket.route	Send ticket to expert

Addison then considered the survey functionality. Working with Sydney, Addison found that the survey functionality rarely changed and was not overly complicated. Sydney talked with Skyler, the Sysops Squad developer who originally created the `ss.survey.templates` namespace, and found there was no compelling reason to separate the survey templates into their own namespace ("It just seemed like a good idea at the time," said Skyler). With this information, Addison created an architecture story to move the seven class files from `ss.survey.templates` into the `ss.survey` namespace and removed the `ss.survey.template` component, as shown in Table 5-11.

Table 5-11. The prior Sysops Squad Survey components flattened into a single component

Component	Namespace	Responsibility
Survey	ss.survey	Send and receive surveys

After applying the Flatten Components pattern (illustrated in Figure 5-12), Addison observed that there were no "hills" (component upon component) or orphaned classes and that all of the components were contained only in the leaf nodes of the corresponding namespace.

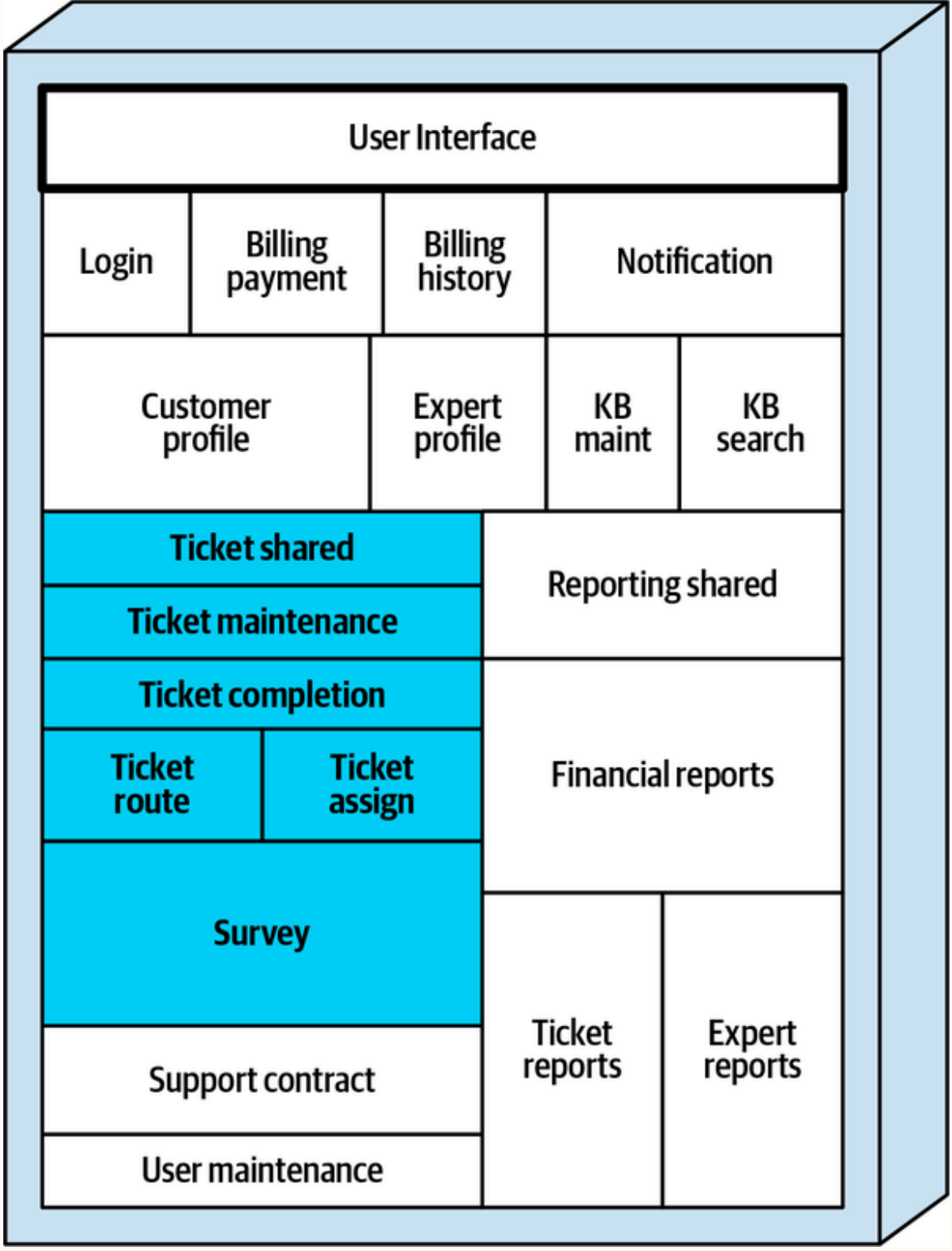


Figure 5-12. The Survey component was flattened into a single component, whereas the Ticket component was raised up and flattened, creating a Ticket subdomain

Addison recorded the results of the refactoring efforts thus far in applying these decomposition patterns and listed them in Table 5-12.

Table 5-12. Sysops Squad components after applying the Flatten Components pattern

Component	Namespace
Login	ss.login
Billing Payment	ss.billing.payment
Billing History	ss.billing.history
Customer Profile	ss.customer.profile
Expert Profile	ss.expert.profile
KB Maint	ss.kb.maintenance
KB Search	ss.kb.search
Notification	ss.notification
Reporting Shared	ss.reporting.shared
Ticket Reports	ss.reporting.tickets
Expert Reports	ss.reporting.experts
Financial Reports	ss.reporting.financial
Ticket Shared	ss.ticket.shared
Ticket Maintenance	ss.ticket.maintenance
Ticket Completion	ss.ticket.completion
Ticket Assign	ss.ticket.assign
Ticket Route	ss.ticket.route
Support Contract	ss.supportcontract

Component	Namespace
Survey	ss.survey
User Maintenance	ss.users

Determine Component Dependencies Pattern

Three of the most common questions asked when considering a migration from a monolithic application to a distributed architecture are as follows:

1. Is it feasible to break apart the existing monolithic application?
2. What is the rough overall level of effort for this migration?
3. Is this going to require a rewrite of the code or a refactoring of the code?

One of your authors was engaged several years ago in a large migration effort to move a complex monolithic application to microservices. On the first day of the project, the CIO wanted to know only one thing—was this migration effort a golfball, basketball, or an airliner? Your author was curious about the sizing comparisons, but the CIO insisted that the answer to this simple question shouldn't be that difficult given that kind of coarse-grained sizing. As it turned out, applying the *Determine Component Dependencies* pattern quickly and easily answered this question for the CIO—the effort was unfortunately an airliner, but only a small Embraer 190 migration rather than a large Boeing 787 Dreamliner migration.

Pattern Description

The purpose of the *Determine Component Dependencies* pattern is to analyze the incoming and outgoing dependencies (coupling) between *components* to determine what the resulting service dependency graph might look like after breaking up the monolithic application. While there are many factors in determining the right level of granularity for a service (see Chapter 7), each component in the monolithic application is potentially a service candidate (depending on the target distributed architecture style). For this reason, it is critical to understand the interactions and dependencies between components.

It's important to note that this pattern is about component dependencies, not individual class dependencies within a component. A *component dependency* is formed when a class from one component (namespace) interacts with a class from another component (namespace). For example, suppose the `CustomerSurvey` class in the `ss.survey` component in-

vokes a method in the `CustomerNotification` class in the `ss.notification` component to send out the customer survey, as illustrated in the pseudocode in Example 5-7.

Example 5-7. Pseudocode showing a dependency between the Survey and Notification components

```
namespace ss.survey
class CustomerSurvey {
  function createSurvey {
    ...
  }

  function sendSurvey {
    ...
    ss.notification.CustomerNotification.send(customer_id, survey)
  }
}
```

Notice the dependency between the Survey and Notification components, because the `CustomerNotification` class used by the `CustomerSurvey` class resides outside the `ss.survey` namespace. Specifically, the Survey component would have an efferent (or outgoing) dependency on the Notification component, and the Notification component would have an afferent (or incoming) dependency on the Survey component.

Note that the classes *within* a particular component may be a highly coupled mess of numerous dependencies, but that doesn't matter when applying this pattern—what matters is only those dependencies *between* components.

Several tools are available that can assist in applying this pattern and visualizing component dependencies. In addition, many modern IDEs have plug-ins that will produce dependency diagrams of the components, or namespaces, within a particular codebase. These visualizations can be useful in answering the three key questions posed at the start of this section.

For example, consider the dependency diagram shown in Figure 5-13, where the boxes represent components (not classes), and the lines represent coupling points between the components. Notice there is only a single dependency between the components in this diagram, making this application a good candidate for breaking apart since the components are functionally independent from one another.

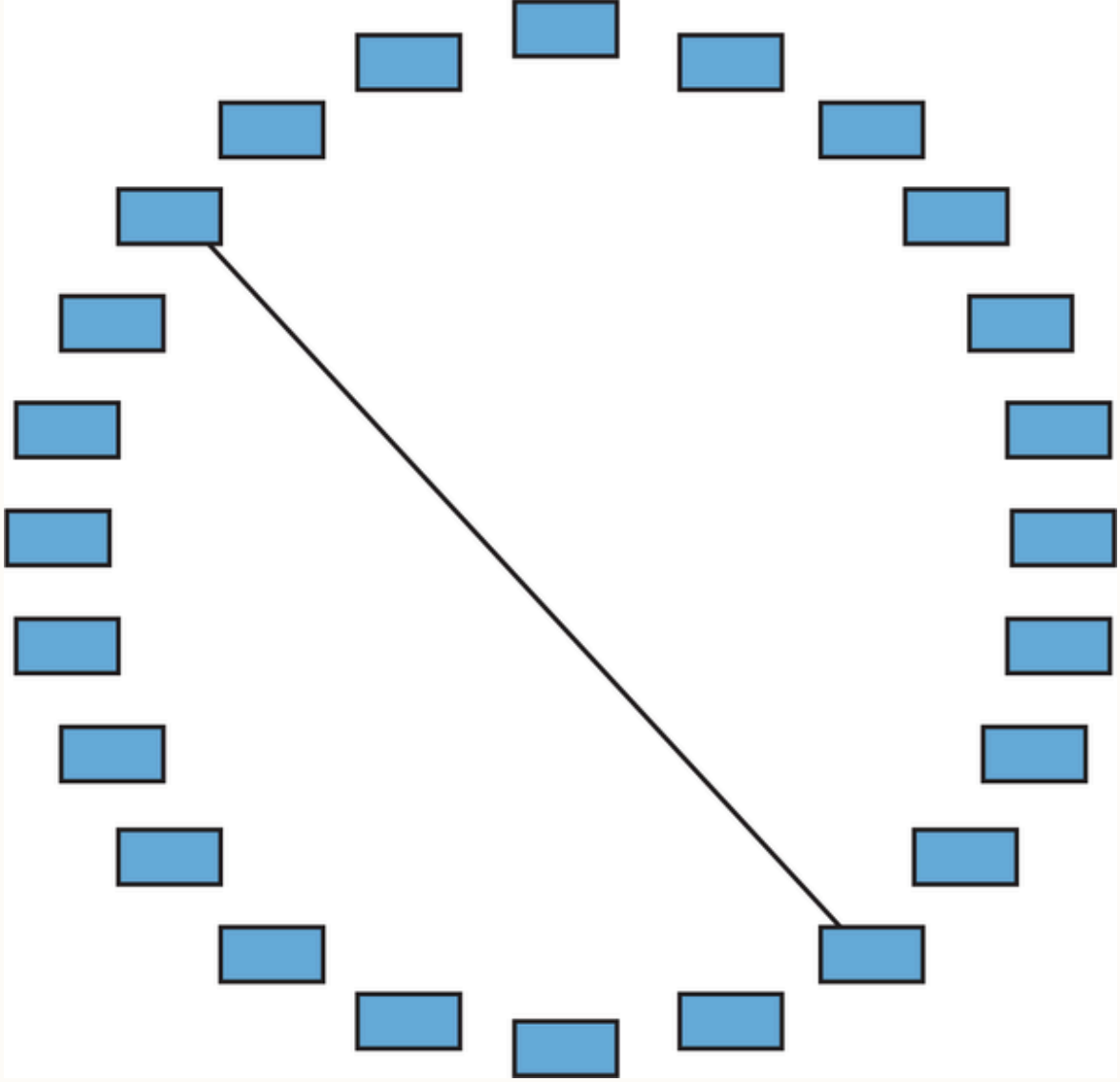


Figure 5-13. A monolithic application with minimal component dependencies takes less effort to break apart (golf ball sizing)

With a dependency diagram like Figure 5-13, the answers to the three key questions are as follows:

1. Is it feasible to break apart the existing monolithic application? *Yes*
2. What is the rough overall level of effort for this migration? *A golf ball (relatively straightforward)*
3. Is this going to be a rewrite of the code or a refactoring of the code? *Refactoring (moving existing code into separately deployed services)*

Now look at the dependency diagram shown in Figure 5-14. Unfortunately, this diagram is typical of the dependencies between components in most business applications. Notice in particular how the lefthand side of this diagram has the highest level of coupling, whereas the righthand side looks much more feasible to break apart.

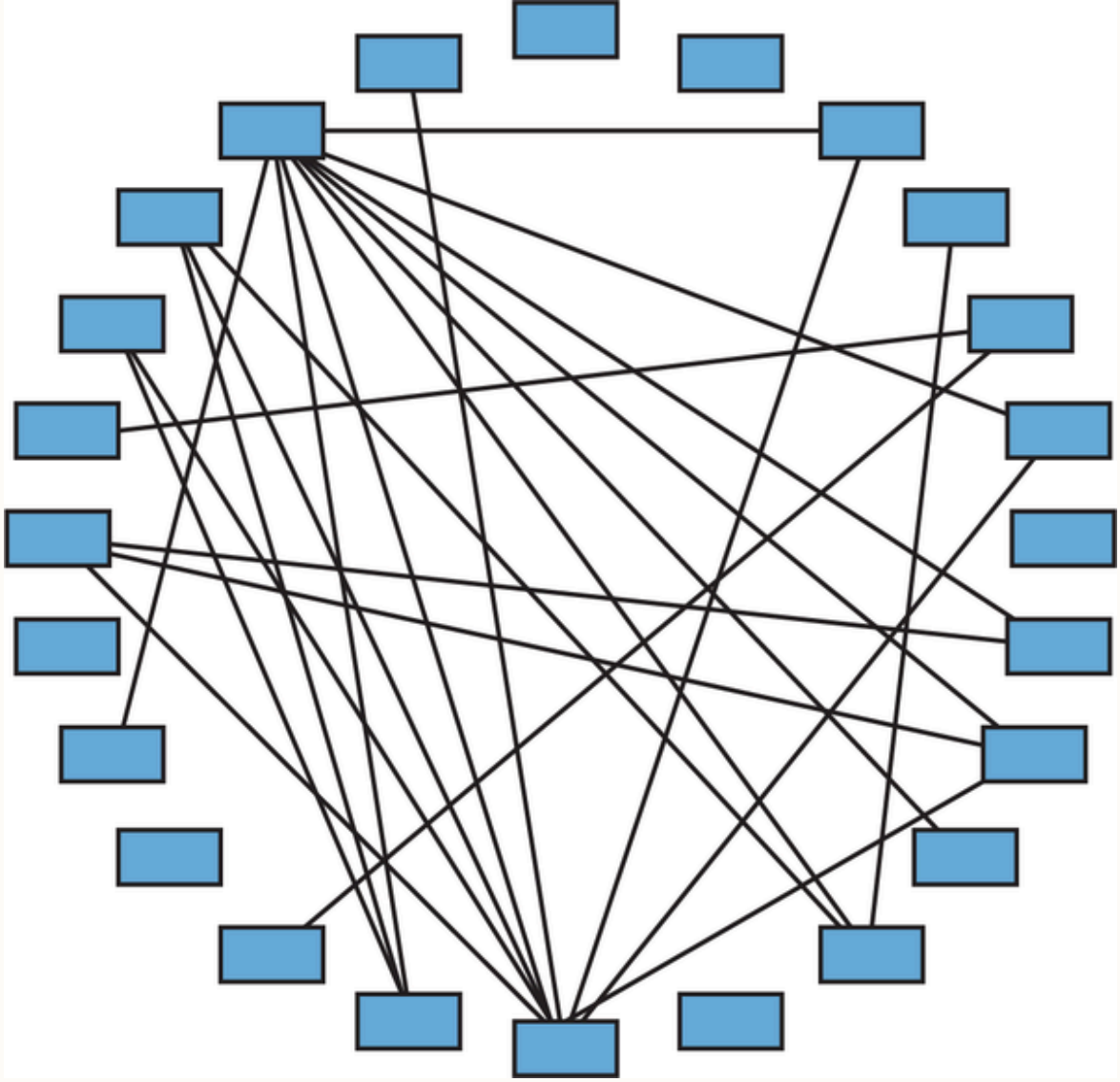


Figure 5-14. A monolithic application with a high number of component dependencies takes more effort to break apart (basketball sizing)

With this level of tight coupling between components, the answers to the three key questions are not very encouraging:

1. Is it feasible to break apart the existing monolithic application? *Maybe...*
2. What is the rough overall level of effort for this migration? *A basketball (much harder)*
3. Is this going to be a rewrite of the code or a refactoring of the code? *Likely a combination of some refactoring and some rewriting of the existing code*

Finally, consider the dependency diagram illustrated in Figure 5-15. In this case, the architect should turn around and run in the opposite direction as fast as they can!

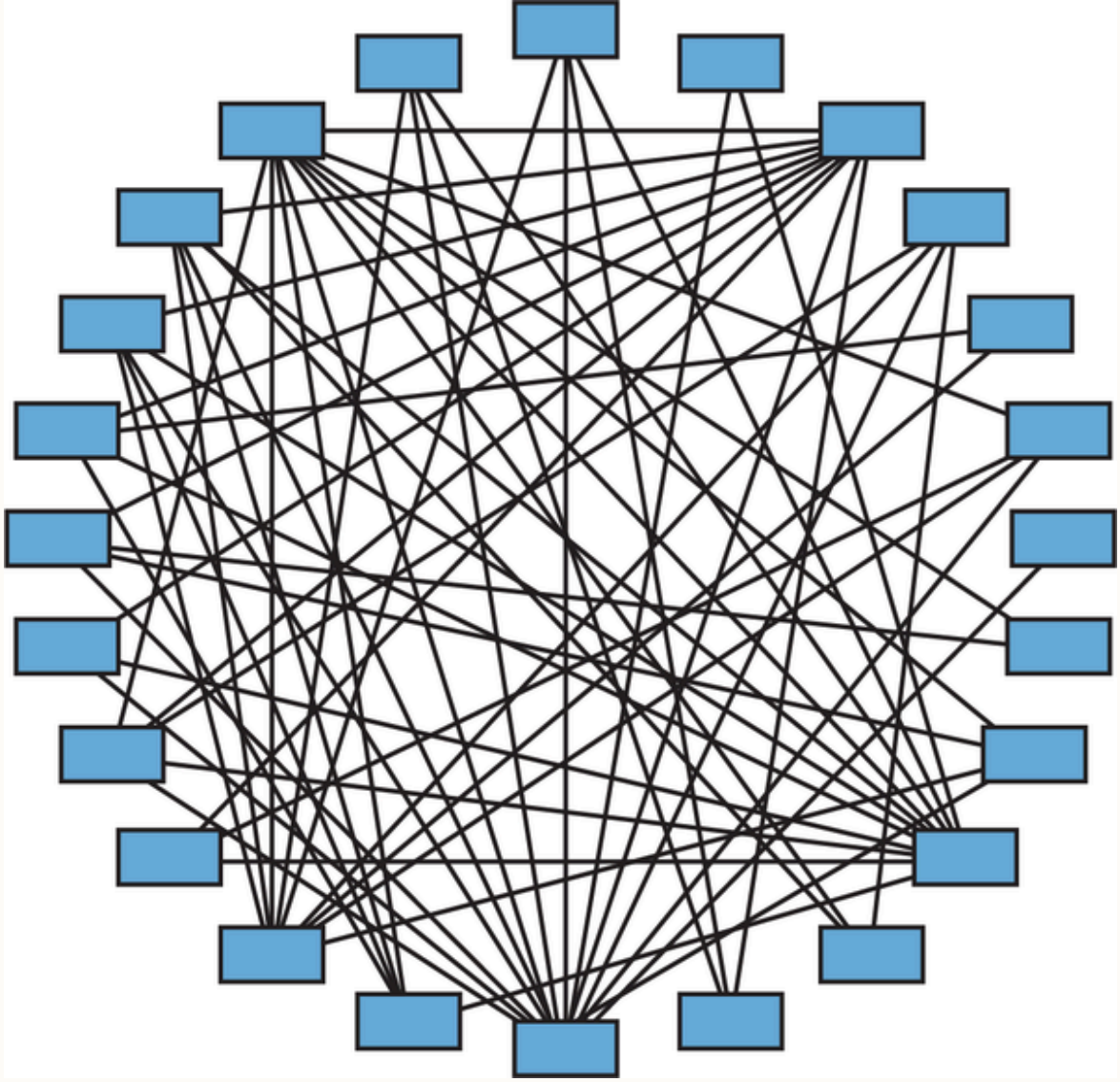


Figure 5-15. A monolithic application with too many component dependencies is not feasible to break apart (airliner sizing)

The answers to the three key questions for applications with this sort of component dependency matrix are not surprising:

1. Is it feasible to break apart the existing monolithic application? *No*
2. What is the rough overall level of effort for this migration? *An airliner*
3. Is this going to be a rewrite of the code or a refactoring of the code? *Total rewrite of the application*

We cannot stress enough the importance of these kinds of visual diagrams when breaking apart a monolithic application. In essence these diagrams form a *radar* from which to determine where the enemy (high component coupling) is located, and also paint a picture of what the resulting service dependency matrix might look like if the monolithic application were to be broken into a highly distributed architecture.

It has been our experience that component coupling is one of the most significant factors in determining the overall success (and feasibility) of a monolithic migration effort. Identifying and understanding the level of component coupling not only allows the architect to determine the feasibility of the migration effort, but also what to expect in terms of

the overall level of effort. Unfortunately, all too often we see teams jump straight into breaking a monolithic application into microservices without having any analysis or visuals into what the monolithic application even looks like. And not surprisingly, those teams struggle to break apart their monolithic applications.

This pattern is useful not only for identifying the overall level of component coupling in an application, but also for determining dependency refactoring opportunities prior to breaking apart the application. When analyzing the coupling level between components, it is important to analyze both afferent (incoming) coupling (denoted in most tools as *CA*), and efferent (outgoing) coupling (denoted in most tools as *CE*). *CT*, or total coupling, is the sum of both afferent and efferent coupling.

Many times, breaking apart a component can reduce the level of coupling of that component. For example, assume component A has an afferent coupling level of 20 (meaning, 20 other components are dependent on the functionality of the component). This does not necessarily mean that all 20 of the other components require *all* of the functionality from component A. Maybe 14 of the other components require only a small part of the functionality contained in component A. Breaking component A into two different components (component A1 containing the smaller, coupled functionality, and component A2 containing the majority of the functionality) reduces the afferent coupling in component A2 to 6, with component A1 having an afferent coupling level of 14.

Fitness Functions for Governance

Two ways to automate the governance for component dependencies are to make sure no component has "too many" dependencies, and to restrict certain components from being coupled to other components. The fitness functions described next are some ways of governing these type of dependencies.

Fitness function: No component shall have more than <some number> of total dependencies

This automated holistic fitness function can be triggered on deployment through a CI/CD pipeline to make sure that the coupling level of any given component doesn't exceed a certain threshold. It is up to the architect to determine that this maximum threshold should be based on the overall level of coupling within the application and the number of components. An alert generated from this fitness function allows the architect to discuss any sort of increase in coupling with the development team, possibly promoting action to break apart components to reduce coupling. This fitness function could also be modified to generate an alert for a threshold limit of incoming only, outgoing only, or both (as separate fitness functions). Example 5-8 shows the pseudocode for sending an alert if the total cou-

pling (incoming and outgoing) exceeds a combined level of 15, which for most applications would be considered relatively high.

Example 5-8. Pseudocode for limiting the total number of dependencies of any given component

```
# Walk the directory structure, gathering components and the source code files
# contained within those components
LIST component_list = identify_components(root_directory)
MAP component_source_file_map
FOREACH component IN component_list {
  LIST component_source_file_list = get_source_files(component)
  ADD component, component_source_file_list TO component_source_file_map
}

# Determine how many references exist for each source file and send an alert if
# the total dependency count is greater than 15
FOREACH component,component_source_file_list IN component_source_file_map {
  FOREACH source_file IN component_source_file_list {
    incoming count = used_by_other_components(source_file, component_source_file_map) {
    outgoing_count = uses_other_components(source_file) {
    total_count = incoming count + outgoing count
  }
  IF total_count > 15 {
    send_alert(component, total_count)
  }
}
```

Fitness function: <some component> should not have a dependency on <another component>

This automated holistic fitness function can be triggered on deployment through a CI/CD pipeline to restrict certain components from having a dependency on other ones. In most cases, there will be one fitness function for each dependency restriction so that, if there were 10 different component restrictions, there would be 10 different fitness functions, one for each component in question. Example 5-9 shows an example using ArchUnit for ensuring that the Ticket Maintenance component (`ss.ticket.maintenance`) does not have a dependency on the Expert Profile component (`ss.expert.profile`).

Example 5-9. ArchUnit code for governing dependency restrictions between components

```
public void ticket_maintenance_cannot_access_expert_profile() {
  noClasses().that()
  .resideInAPackage("..ss.ticket.maintenance..")
```

```
    .should().accessClassesThat()
    .resideInAPackage("..ss.expert.profile..")
    .check(myClasses);
  }
```

Sysops Squad Saga: Identifying Component Dependencies

Monday, November 15, 09:45



After reading about the Determine Component Dependencies pattern, Addison wondered what the Sysops Squad application dependency matrix looked like and whether it was feasible to even break the application apart. Addison used an IDE plug-in to generate a component dependency diagram of the current Sysops Squad application. Initially, Addison felt a bit discouraged because Figure 5-16 showed a lot of dependencies between the Sysops Squad application components.

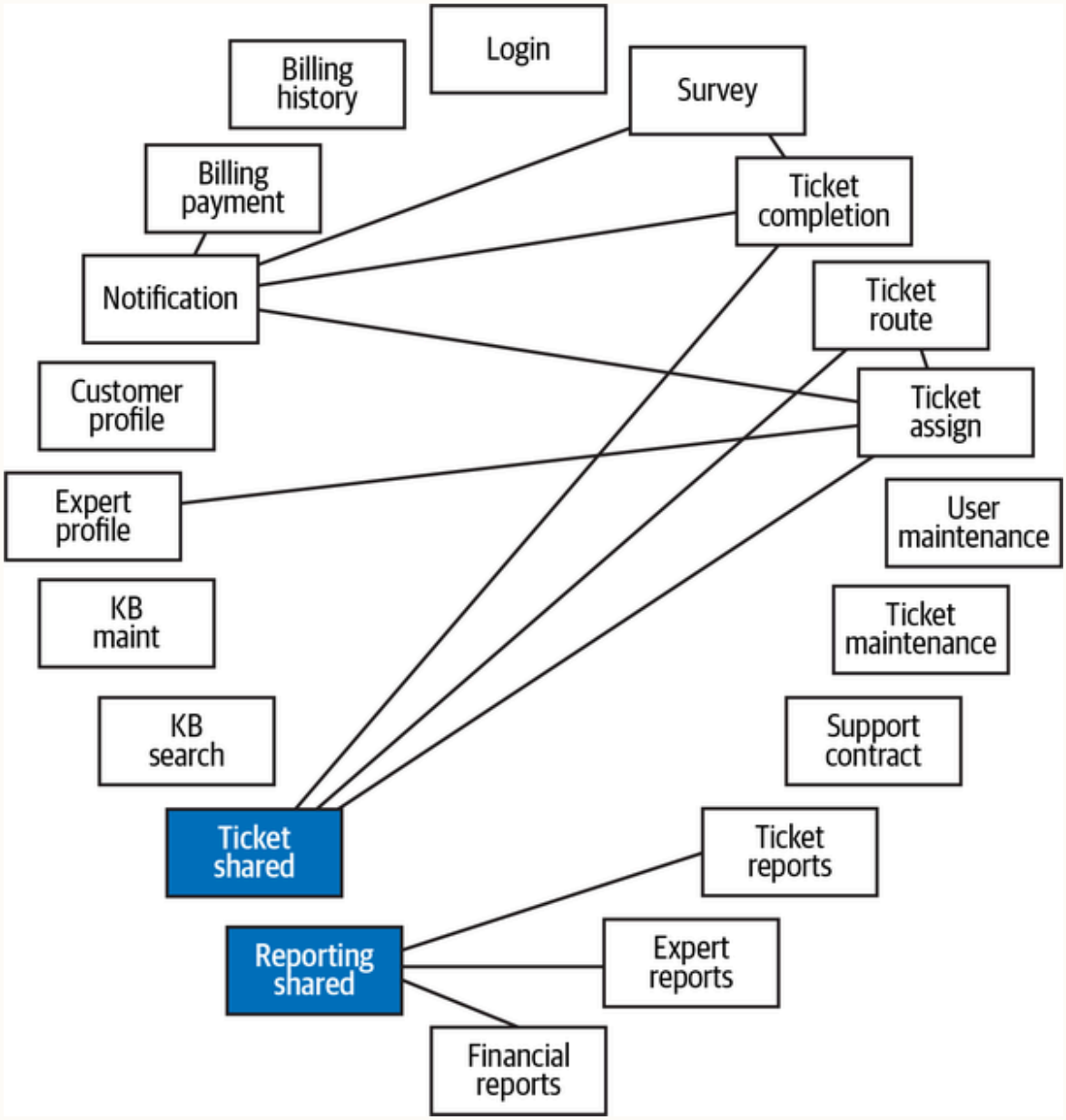


Figure 5-16. Component dependencies in the Sysops Squad application

However, after further analysis, Addison saw that the Notification component had the most dependencies, which was not surprising given that it's a shared component. However, Addison also saw lots of dependencies within the Ticketing and Reporting components. Both of these domain areas have a specific component for shared code (interfaces, helper classes, entity classes, and so on). Realizing that both the ticketing and reporting shared code contains mostly compile-based class references and would likely be implemented as shared libraries rather than services, Addison filtered out these components to get a better view of the dependencies between the core functionality of the application, which is illustrated in Figure 5-17.

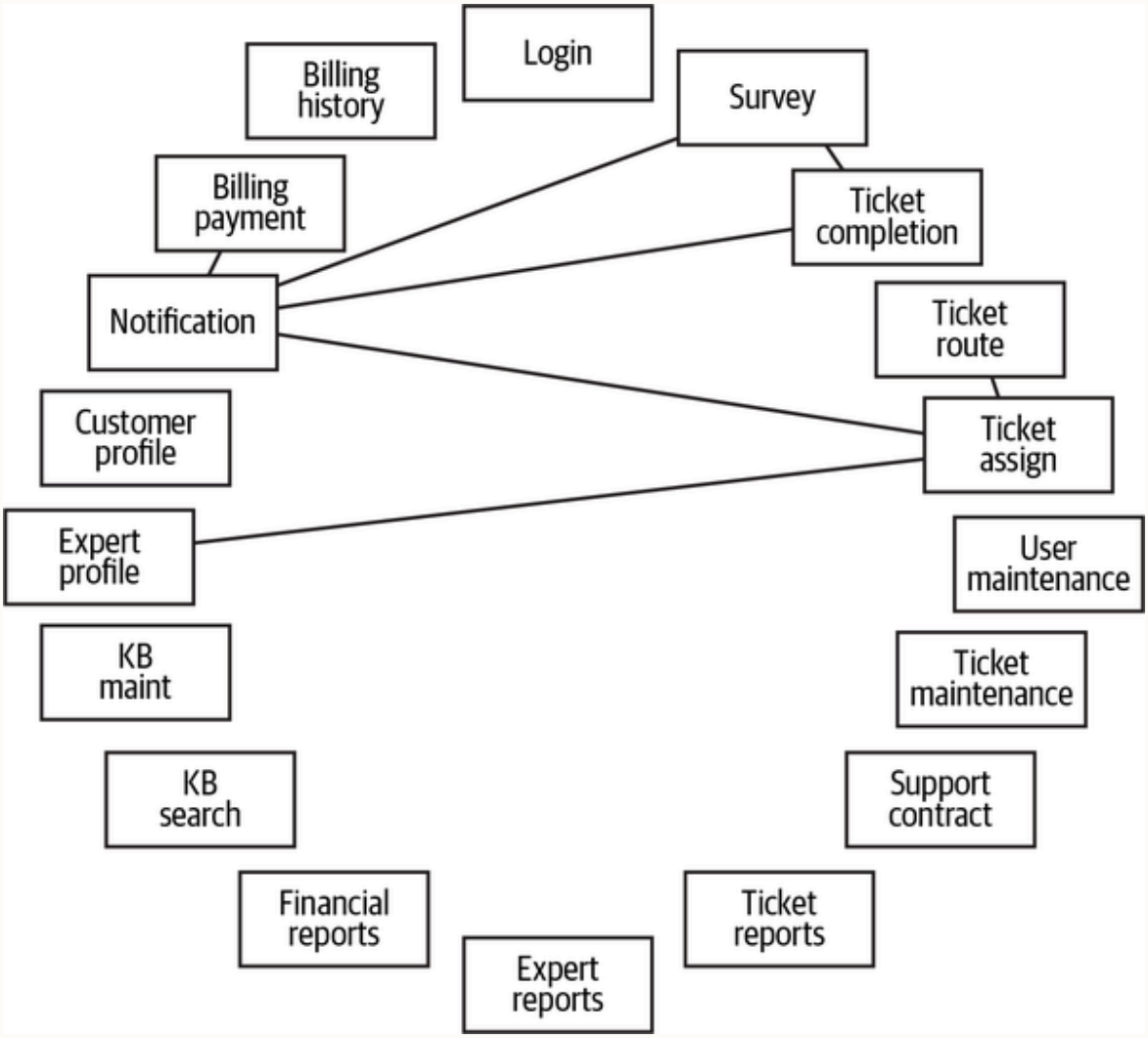


Figure 5-17. Component dependencies in the Sysops Squad application without shared library dependencies

With the shared components filtered out, Addison saw that the dependencies were fairly minimal. Addison showed these results to Austen, and they both agreed that most of the components were relatively self-contained and it appeared that the Sysops Squad application was a good candidate for breaking apart into a distributed architecture.

Create Component Domains Pattern

While each component identified within a monolithic application can be considered a possible candidate for a separate service, in most cases the relationship between a service and components is a one-to-many relationship—that is, a single service may contain one or more components. The purpose of the *Create Component Domains* pattern is to logically group components together so that more coarse-grained domain services can be created when breaking up an application.

Pattern Description

Identifying component domains—the grouping of components that perform some sort of related functionality—is a critical part of breaking apart any monolithic application. Recall the advice from Chapter 4:

> *When breaking apart monolithic applications, consider first moving to service-based architecture as a stepping-stone to other distributed architectures.*

Creating component domains is an effective way of determining what will eventually become domain services in a service-based architecture.

Component domains are physically manifested in an application through component namespaces (or directories). Because namespace nodes are hierarchical in nature, they become an excellent way of representing the domains and subdomains of functionality. This technique is illustrated in Figure 5-18, where the second node in the namespace (`.customer`) refers to the *domain*, the third node represents a *subdomain* under the customer domain (`.billing`), and the leaf node (`.payment`) refers to the *component*. The `.MonthlyBilling` at the end of this namespace refers to a class file contained within the Payment component.

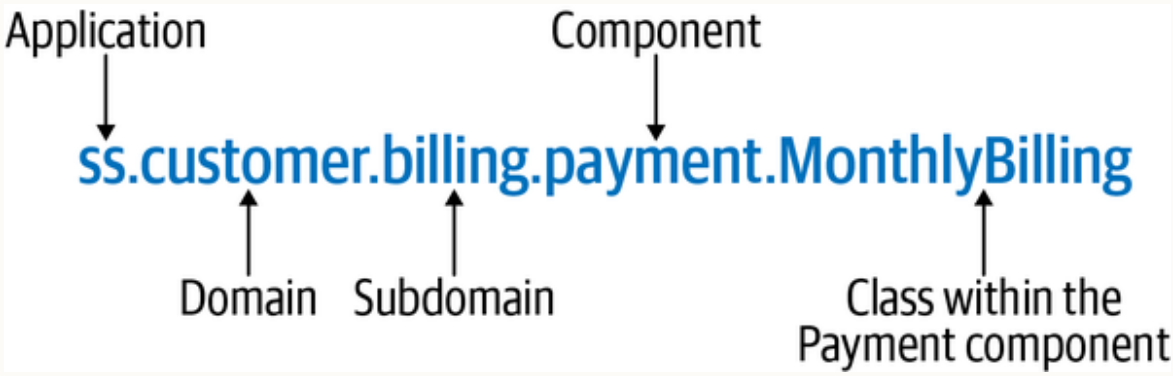


Figure 5-18. Component domains are identified through the namespace nodes

Since many older monolithic applications were implemented prior to the widespread use of domain-driven design, in many cases refactoring of the namespaces is needed to struc-

turally identify domains within the application. For example, consider the components listed in Table 5-13 that make up the Customer domain within the Sysops Squad application.

Table 5-13. Components related to the Customer domain before refactoring

Component	Namespace
Billing Payment	ss.billing.payment
Billing History	ss.billing.history
Customer Profile	ss.customer.profile
Support Contract	ss.supportcontract

Notice how each component is related to customer functionality, but the corresponding namespaces don't reflect that association. To properly identify the Customer domain (manifested through the namespace `ss.customer`), the namespaces for the Billing Payment, Billing History, and Support Contract components would have to be modified to add the `.customer` node at the beginning of the namespace, as shown in Table 5-14.

Table 5-14. Components related to the Customer domain after refactoring

Component	Namespace
Billing Payment	ss.customer.billing.payment
Billing History	ss.customer.billing.history
Customer Profile	ss.customer.profile
Support Contract	ss.customer.supportcontract

Notice in the prior table that all of the customer-related functionality (billing, profile maintenance, and support contract maintenance) is now grouped under `.customer`, aligning each component with that particular domain.

Fitness Functions for Governance

Once refactored, it's important to govern the component domains to ensure that namespace rules are enforced and that no code exists outside the context of a component domain or subdomain. The following automated fitness function can be used to help govern component domains once they are established within the monolithic application.

Fitness function: All namespaces under <root namespace node> should be restricted to <list of domains>

This automated holistic fitness function can be triggered on deployment through a CI/CD pipeline to restrict the domains contained within an application. This fitness function helps prevent additional domains from being inadvertently created by development teams and alerts the architect if any new namespaces (or directories) are created outside the approved list of domains. Example 5-10 shows an example using ArchUnit for ensuring that only the ticket, customer, and admin domains exist within an application.

Example 5-10. ArchUnit code for governing domains within an application

```
public void restrict_domains() {
  classes()
    .should().resideInAPackage("..ss.ticket..")
    .orShould().resideInAPackage("..ss.customer..")
    .orShould().resideInAPackage("..ss.admin..")
    .check(myClasses);
}
```

Sysops Squad Saga: Creating Component Domains

Thursday, November 18, 13:15



Addison and Austen consulted with Parker, the Sysops Squad product owner, and together they identified five main domains within the application: a Ticketing domain (`ss.ticket`) containing all ticket-related functionality, including ticket processing, customer surveys, and knowledge base (KB) functionality; a Reporting domain (`ss.reporting`) containing all reporting functionality; a Customer domain (`ss.customer`) containing customer profile, billing, and support contracts; an Admin domain (`ss.admin`) containing maintenance of users and Sysops Squad experts; and finally, a Shared domain (`ss.shared`) containing login and notification functionality used by the other domains.

Addison created a domain diagram (see Figure 5-19) showing the various domains and corresponding groups of components within each domain, and was satisfied with this grouping

as no component was left out, and there was good cohesion between the components within each domain.

The exercise Addison did in diagramming and grouping the components was an important one as it validated the identified domain candidates and also demonstrated the need for collaboration with business stakeholders (such as the product owner or business application sponsor). Had the components not lined up properly or Addison was left with components that didn't belong anywhere, more collaboration with Parker (the product owner) would have been necessary.

Satisfied that all of the components fit nicely into these domains, Addison then looked at the various component namespaces in Table 5-12 after applying the "Flatten Components Pattern" and identified the component domain refactoring that needed to take place.

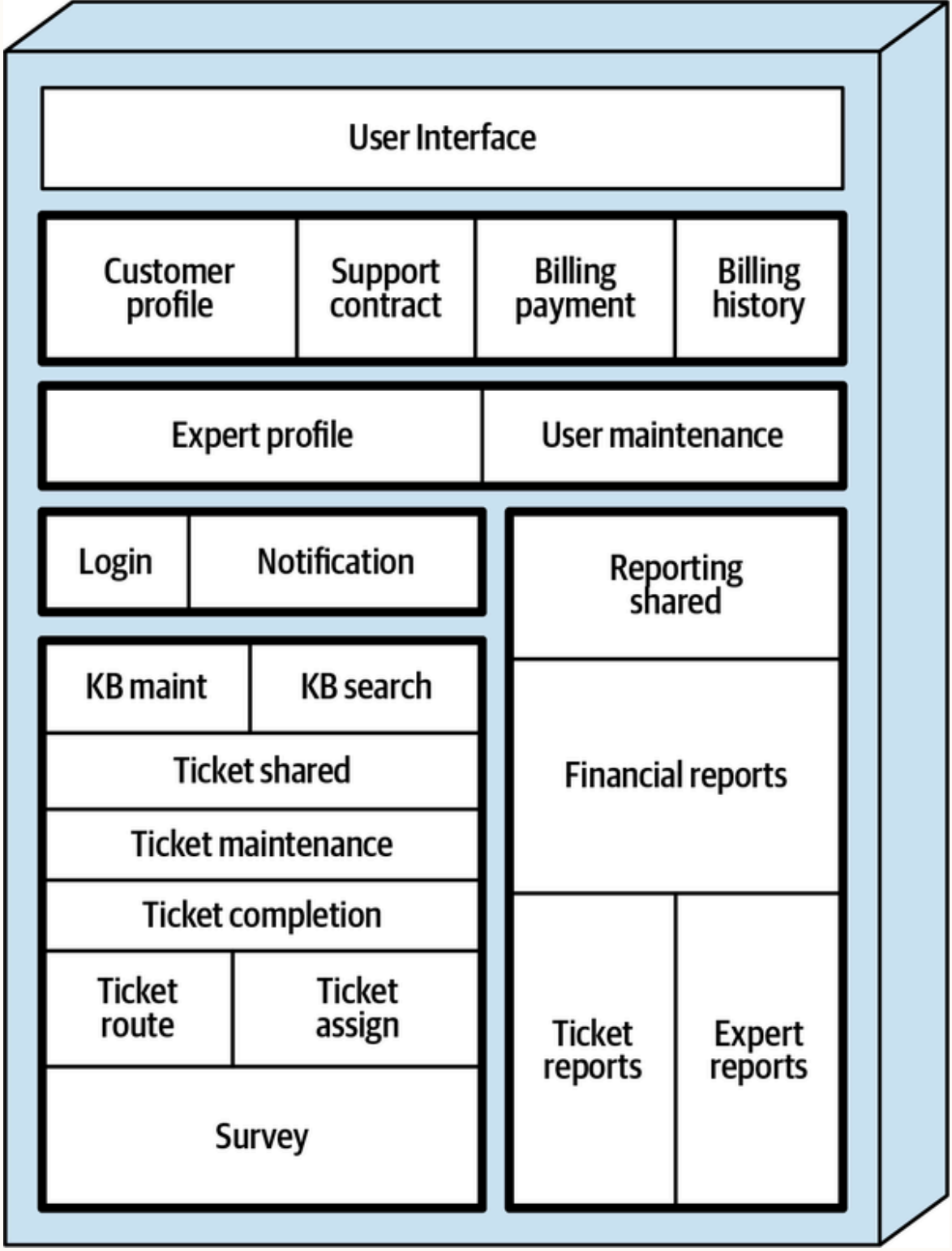


Figure 5-19. The five domains identified (with darkened borders) within the Sysops Squad application

Addison started with the Ticket domain and saw that while the core ticket functionality started with the namespace `ss.ticket`, the survey and knowledge base components did not. Therefore, Addison wrote an architecture story to refactor the components listed in Table 5-15 to align with the ticketing domain.

Table 5-15. Sysops Squad component refactoring for the Ticket domain

Component	Domain	Current namespace	Target namespace
KB Maint	Ticket	ss.kb.maintenance	ss.ticket.kb.maintenance
KB Search	Ticket	ss.kb.search	ss.ticket.kb.search
Ticket Shared	Ticket	ss.ticket.shared	Same (no change)
Ticket Maintenance	Ticket	ss.ticket.maintenance	Same (no change)
Ticket Completion	Ticket	ss.ticket.completion	Same (no change)
Ticket Assign	Ticket	ss.ticket.assign	Same (no change)
Ticket Route	Ticket	ss.ticket.route	Same (no change)
Survey	Ticket	ss.survey	ss.ticket.survey

Next Addison considered the customer-related components, and found that the billing and survey components needed to be refactored to include them under the Customer domain, creating a Billing subdomain in the process. Addison wrote an architecture story for the refactoring of the Customer domain functionality, shown in Table 5-16.

Table 5-16. Sysops Squad component refactoring for the Customer domain

Component	Domain	Current namespace	Target namespace
Billing Payment	Customer	ss.billing.payment	ss.customer.billing.payment
Billing History	Customer	ss.billing.history	ss.customer.billing.history
Customer Profile	Customer	ss.customer.profile	Same (no change)
Support Contract	Customer	ss.supportcontract	ss.customer.supportcontract

By applying the "Identify and Size Components Pattern", Addison found that the reporting domain was already aligned, and no further action was needed with the reporting components listed in Table 5-17.

Table 5-17. Sysops Squad Reporting components are already aligned with the Reporting domain

Component	Domain	Current namespace	Target namespace
Reporting Shared	Reporting	ss.reporting.shared	Same (no change)
Ticket Reports	Reporting	ss.reporting.tickets	Same (no change)
Expert Reports	Reporting	ss.reporting.experts	Same (no change)
Financial Reports	Reporting	ss.reporting.financial	Same (no change)

Addison saw that both the Admin and Shared domains needed alignment as well, and decided to create a single architecture story for this refactoring effort and listed these components in Table 5-18. Addison also decided to rename the `ss.expert.profile` namespace to `ss.experts` to avoid an unnecessary Expert subdomain under the Admin domain.

Table 5-18. Sysops Squad component refactoring for the Admin and Shared domains

Component	Domain	Current namespace	Target namespace
Login	Shared	ss.login	aa.shared.login
Notification	Shared	ss.notification	ss.shared.notification
Expert Profile	Admin	ss.expert.profile	ss.admin.experts
User Maintenance	Admin	ss.users	ss.admin.users

With this pattern complete, Addison realized they were now prepared to structurally break apart the monolithic application and move to the first stage of a distributed architecture by applying the Create Domain Services pattern (described in the next section).

Create Domain Services Pattern

Once components have been properly sized, flattened, and grouped into domains, those domains can then be moved to separately deployed *domain services*, creating what is known as a service-based architecture (see Appendix A). Domain services are coarse-grained, separately deployed units of software containing all of the functionality for a particular domain (such as Ticketing, Customer, Reporting, and so on).

Pattern Description

The previous "Create Component Domains Pattern" forms well-defined component domains within a monolithic application and manifests those domains through the component namespaces (or directory structures). This pattern takes those well-defined component domains and extracts those component groups into separately deployed services, known as a *domain services*, thus creating a service-based architecture.

In its simplest form, service-based architecture consists of a user interface that remotely accesses coarse-grained domain services, all sharing a single monolithic database. Although there are many topologies within service-based architecture (such as breaking up the user interface, breaking up the database, adding an API gateway, and so on), the basic topology shown in Figure 5-20 is a good starting point for migrating a monolithic application.

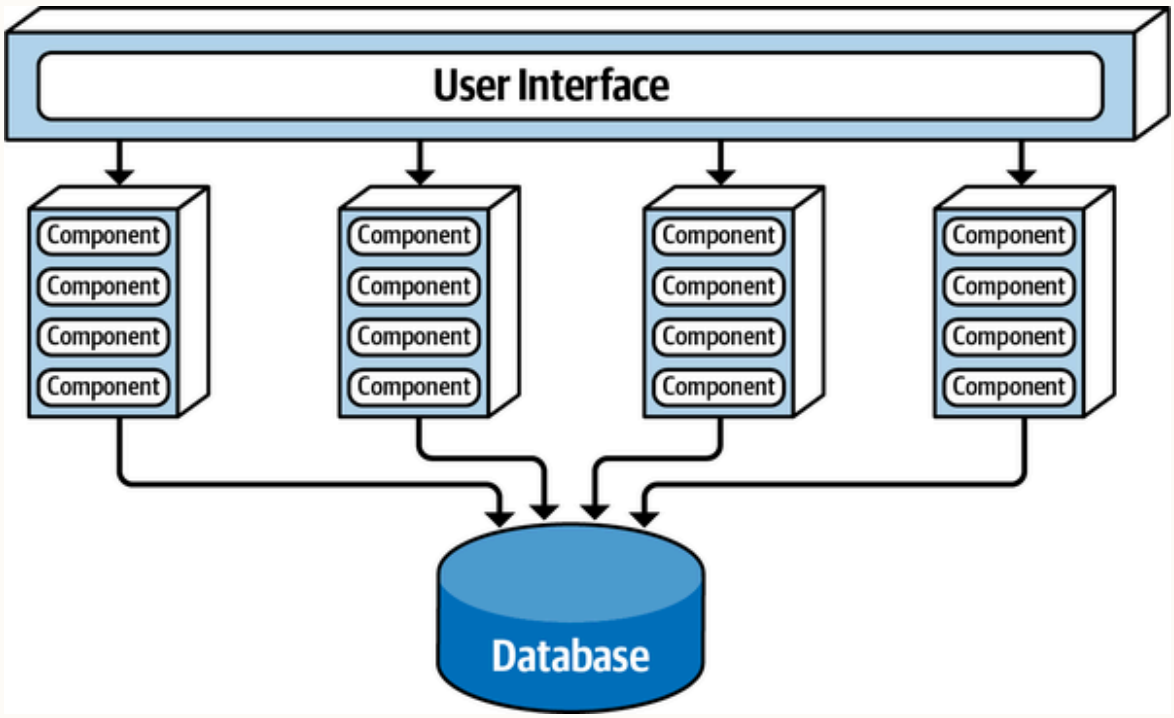


Figure 5-20. The basic topology for a service-based architecture

In addition to the benefits mentioned in "Component-Based Decomposition", moving to service-based architecture first allows the architect and development team to learn more about each domain service to determine whether it should be broken into smaller services

within a microservices architecture or left as a larger domain service. Too many teams make the mistake of starting out too fine-grained, and as a result must embrace all of the trappings of microservices (such as data decomposition, distributed workflows, distributed transactions, operational automation, containerization, and so on) without the need for all of those fine-grained microservices.

Figure 5-21 illustrates how the *Create Domain Services* pattern works. Notice in the diagram how the Reporting component domain defined in the "Create Component Domains Pattern" is extracted from of the monolithic application, forming its own separately deployed Reporting service.

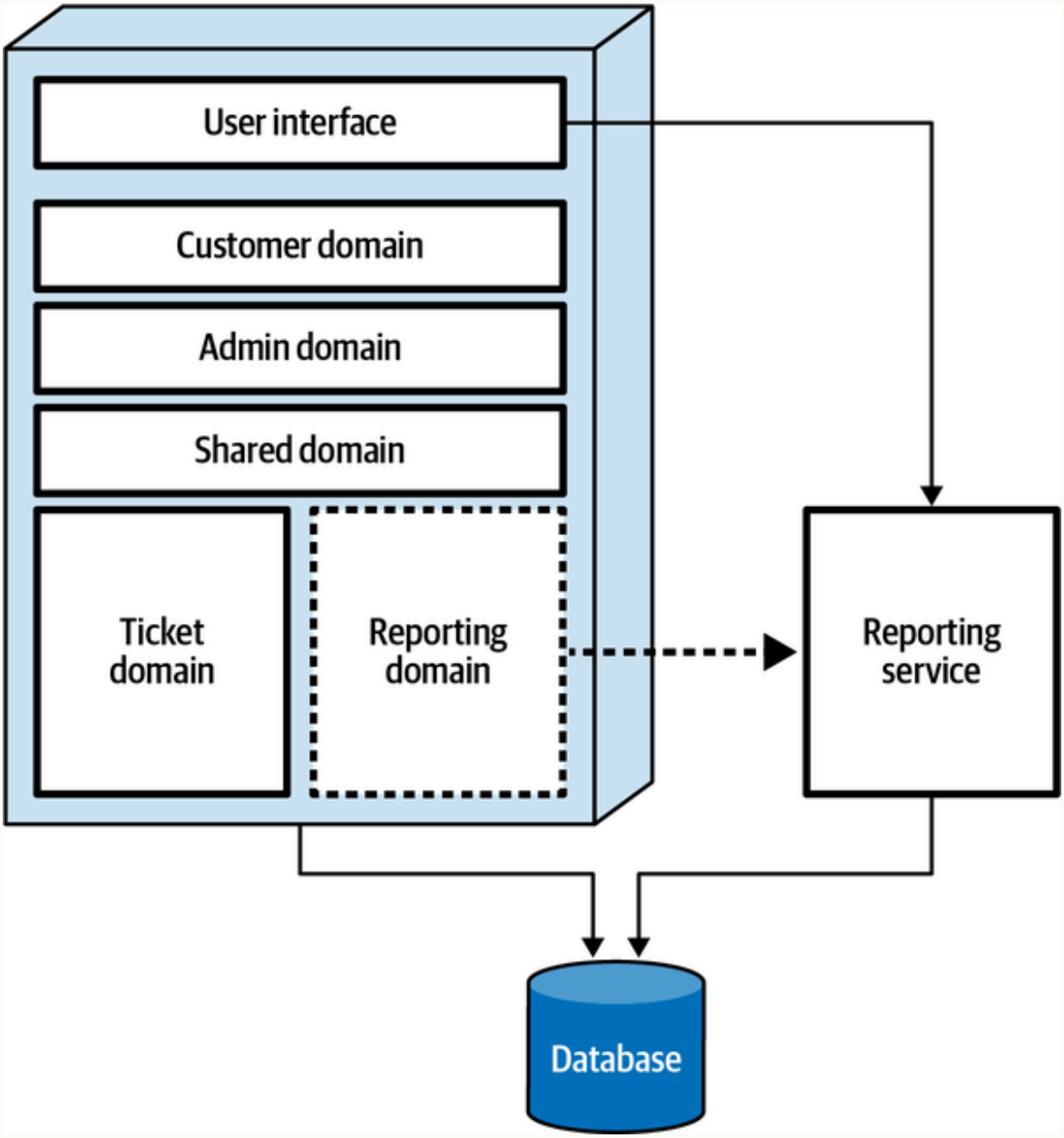


Figure 5-21. Component domains are moved to external domain services

A word of advice, however: don't apply this pattern until *all* of the component domains have been identified and refactored. This helps reduce the amount of modification needed to each domain service when moving components (and hence source code) around. For example, suppose all of the ticketing and knowledge base functionality in the Sysops Squad application was grouped and refactored into a Ticket domain, and a new Ticket service cre-

ated from that domain. Now suppose that the customer survey component (identified through the `ss.customer.survey` namespace) was deemed part of the Ticket domain. Since the Ticket domain had already been migrated, the Ticket service would now have to be modified to include the Survey component. Better to align and refactor all of the components into component domains first, *then* start migrating those component domains to domain services.

Fitness Functions for Governance

It is important to keep the components within each domain service aligned with the domain, particularly if the domain service will be broken into smaller microservices. This type of governance helps keep domain services from becoming their own unstructured monolithic service. The following fitness function ensures that the namespace (and hence components) are kept consistent within a domain service.

Fitness function: All components in <some domain service> should start with the same namespace

This automated holistic fitness function can be triggered on deployment through a CI/CD pipeline to make sure the namespaces for components within a domain service remain consistent. For example, all components within the Ticket domain service should start with `ss.ticket`. Example 5-11 uses ArchUnit for ensuring this constraint. Each domain service would have its own corresponding fitness function based on its particular domain.

Example 5-11. ArchUnit code for governing components within the Ticket domain service

```
public void restrict_domain_within_ticket_service() {
  classes().should().resideInAPackage("..ss.ticket..")
  .check(myClasses);
}
```

Sysops Squad Saga: Creating Domain Services

Tuesday, November 23, 09:04



Addison and Austen worked closely with the Sysops Squad development team to develop a migration plan to stage the migration from component domains to domain services. They realized this effort not only required the code within each component domain to be extracted from the monolith and moved to a new project workspace, but also for the user interface to now remotely access the functionality within that domain.

Working from the component domains identified previously in Figure 5-19, the team migrated each component, one at a time, eventually arriving at a service-based architecture, as shown in Figure 5-22. Notice how each domain area identified in the previous pattern now becomes a separately deployed service.

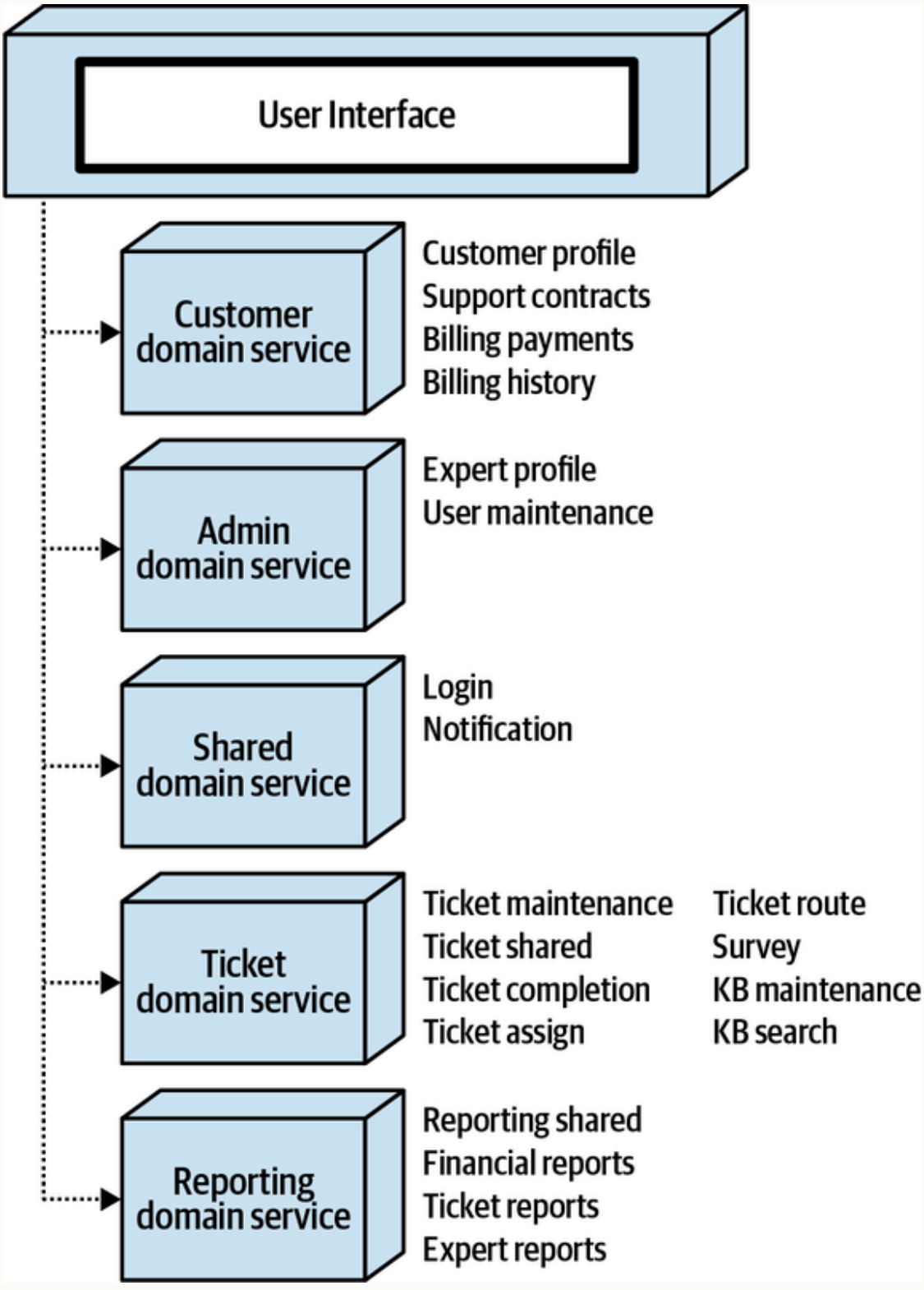


Figure 5-22. Separately deployed domain services result in a distributed Sysops Squad application

Summary

It has been our experience that "seat-of-the-pants" migration efforts rarely produce positive results. Applying these component-based decomposition patterns provides a struc-

tured, controlled, and incremental approach for breaking apart monolithic architectures. Once these patterns are applied, teams can now work to decompose monolithic data (see Chapter 6) and begin breaking apart domain services into more fine-grained microservices (see Chapter 7) as needed.

Previous chapter
< 4. Architectural Decomposition

Next chapter
6. Pulling Apart Operational Data >